

FACULTAD DE INGENIERÍA E INTELIGENCIA ARTIFICIAL

“Implementación de validación léxica, sintáctica y semántica de movimientos de inventario para detectar inconsistencias en una empresa comercial de Lima.”

Curso: Compiladores

Ingeniería de Sistemas de Información

Ingeniería de Software

Ingeniería en Inteligencia Artificial

Estudiante: Muñoz Rosales, Mathias Benjamin | ORCID: 0009-0003-9656-9137

Docente: Delgado Enríquez, Héctor Odín | ORCID: 0000-0003-2112-1349

Lima - 2026

ÍNDICE

AGRADECIMIENTO..... 4
INTRODUCCIÓN AL PROYECTO..... 5
RESUMEN..... 7
DESCRIPCIÓN DEL CASO DE ESTUDIO..... 7
Título del proyecto..... 7
Planteamiento del problema..... 7
Problemática..... 14
Justificación..... 15
Áreas de la empresa..... 16
Procesos principales..... 19
Subprocesos..... 21
Proceso general del negocio..... 25
Organigrama..... 27
OBJETIVOS..... 28
Objetivo general..... 28
Objetivos específicos..... 28
MARCO TEÓRICO..... 30
METODOLOGÍA DE IMPLEMENTACIÓN..... 39
Scrum..... 39
Modelo en cascada (Waterfall)..... 40
Modelo de prototipado..... 41
Justificación de la metodología seleccionada..... 42
DESARROLLO DEL PROYECTO..... 49
1. ANÁLISIS..... 50
1.1. Requerimientos funcionales..... 50
1.2. Requerimientos no funcionales..... 54
1.3. Recursos tecnológicos utilizados..... 58
1.4. Modelamiento formal del proceso..... 59
1.5. Definición del alfabeto..... 61
1.6. Definición de cadenas y lenguaje..... 64
1.7. Expresiones regulares..... 67
1.8. Autómata finito no determinista (AFND)..... 69
1.8.1. AFND para números enteros..... 70
1.8.2. Autómata finito determinista (AFD)..... 71
1.8.3. AFND para cadenas de texto..... 73
1.9. Autómata finito determinista..... 75
1.9.1. AFD para números enteros..... 75
1.9.2. AFD para identificadores..... 77
1.9.3. AFD para cadenas de texto..... 81
2. DISEÑO..... 84
2.1. Diseño de tokens y lexemas..... 84
2.2. Diseño de la gramática libre de contexto..... 84

2.3. Reglas de producción..... 84
2.4. Derivaciones..... 84
2.5. Árbol sintáctico..... 84
3. DESARROLLO..... 84
3.1. Automatización mediante metacompiladores..... 84
3.2. Generación del analizador léxico y sintáctico..... 84
3.3. Implementación de la validación semántica..... 84
3.4. Integración de los componentes..... 84
3.5. Implementación de la interfaz de usuario..... 84
3.6. Despliegue de la aplicación..... 84
4. PRUEBAS..... 84
4.1. Estrategia y casos de prueba..... 84
4.2. Ejecución de pruebas..... 84
4.3. Verificación de resultados y comportamiento del sistema..... 84
DISCUSIÓN..... 85
RESULTADOS..... 85
CONCLUSIONES..... 85
RECOMENDACIONES..... 85
REFERENCIAS..... 85
ANEXOS..... 85

AGRADECIMIENTO

Se agradece a la Universidad San Ignacio de Loyola y a la Facultad de Ingeniería e Inteligencia Artificial por brindar las condiciones académicas necesarias para el desarrollo de este proyecto. De igual manera, se expresa un especial reconocimiento al docente del curso de Compiladores por las orientaciones brindadas para comprender la relación entre la teoría de lenguajes formales, los autómatas finitos y su aplicación en el desarrollo de soluciones computacionales.

INTRODUCCIÓN AL PROYECTO

La gestión de inventarios constituyó una actividad esencial para las empresas comerciales, debido a que la disponibilidad y exactitud de las existencias condicionaron la atención de la demanda, la continuidad de las operaciones y el control de los recursos. El inventario representó uno de los activos relevantes de estas organizaciones y su administración requirió información confiable para orientar las decisiones de reposición, venta y despacho (Corella-Parra & Olea-Miranda, 2023). En este ámbito, la inexactitud de los registros de inventario (IRI) se entendió como la diferencia entre las cantidades físicas y las consignadas en los sistemas de información. Destro et al. (2023) señalaron que estas discrepancias afectaron indicadores operativos, como la productividad del picking, las ventas perdidas y la utilización de la capacidad de los almacenes.

La problemática también se evidenció en investigaciones realizadas en el contexto peruano. Floriano Rodríguez et al. (2024) identificaron que solo el 37,5 % de las micro y pequeñas empresas comerciales estudiadas en Chimbote gestionó sus inventarios de manera eficiente. Por su parte, Vasquez-Layza et al. (2024) reportaron, en una ferretería de Trujillo, una exactitud promedio semanal de inventario de 20 %, además de desorganización del almacén y registros digitales desactualizados. Aunque estos resultados correspondieron a organizaciones y contextos específicos, permitieron reconocer las dificultades asociadas con la confiabilidad de la información de existencias y la necesidad de fortalecer los controles aplicados a los movimientos de inventario.

Los seis antecedentes examinados abordaron estas dificultades desde perspectivas administrativas y operativas. Destro et al. (2023) analizaron la inexactitud de los registros y el conteo cíclico en centros de distribución; Corella-Parra y Olea-Miranda (2023) propusieron un sistema de control de inventario para una empresa comercializadora de sistemas de riego; Jara-Saenz et al. (2026) estudiaron políticas de inventario y pronóstico de demanda en el sector de productos de limpieza; García Peña et al. (2023) analizaron el control interno en negocios mayoristas de abarrotes; Vasquez-Layza et al. (2024) evaluaron mejoras de gestión en una ferretería; y Floriano Rodríguez et al. (2024) examinaron la relación entre el control interno y la gestión de inventarios en MYPE comerciales. En conjunto, estos

trabajos consideraron herramientas como el conteo cíclico, la clasificación ABC, los sistemas de registro, la reorganización del almacén, el pronóstico y el fortalecimiento del control interno. Sin embargo, en los estudios revisados no se identificó una evaluación de un mecanismo formal que verificara preventivamente la estructura y el significado de cada instrucción de movimiento antes de aceptarla. Este vacío delimitó el problema abordado en el proyecto.

En consecuencia, se planteó la siguiente pregunta de investigación: ¿cómo podía implementarse una validación léxica, sintáctica y semántica de movimientos de inventario para detectar operaciones inconsistentes en el contexto de una empresa comercial de Lima? La investigación se desarrolló mediante un caso de estudio hipotético y datos simulados, por lo que no se afirmó que la solución hubiera sido aplicada ni que hubiera producido mejoras en una empresa real. Su justificación fue teórica, al vincular conceptos de lenguajes formales y técnicas de compilación con la representación de movimientos de inventario (Hopcroft et al., 2007); práctica, al proponer un filtro previo a la aceptación de instrucciones; metodológica, al organizar la validación en fases léxica, sintáctica y semántica; y tecnológica, al integrar herramientas de generación de analizadores y desarrollo web.

El objetivo general consistió en implementar una validación léxica, sintáctica y semántica de movimientos de inventario para detectar inconsistencias en el caso hipotético de una empresa comercial de Lima. Para alcanzarlo, se identificaron los errores relevantes de los movimientos, se formalizó un lenguaje específico de dominio (DSL), se definieron sus componentes léxicos y gramaticales, se generaron el analizador léxico y el sintáctico, y se integraron las reglas de validación semántica con el procesamiento de lotes y una interfaz web. Asimismo, se examinó el comportamiento del analizador mediante casos manuales de regresión. La solución se desarrolló con ANTLR 4.13.2 para generar el lexer y el parser, Python para implementar el procesamiento y las validaciones, y Flask para ofrecer una interfaz de interacción. El procesamiento consideró una copia temporal del inventario y archivos JSON como fuentes de datos, sin actualizar de manera definitiva el archivo original de existencias. De este modo, el trabajo se concentró en un analizador o procesador DSL especializado para revisar instrucciones de inventario y emitir aceptación o diagnósticos, sin generar código máquina ni reemplazar un sistema empresarial de gestión de inventarios.

RESUMEN

DESCRIPCIÓN DEL CASO DE ESTUDIO

Título del proyecto

Implementación de validación léxica, sintáctica y semántica de movimientos de inventario para detectar inconsistencias en una empresa comercial de Lima

Planteamiento del problema

Destro et al. (2023), en el artículo científico *The impacts of inventory record inaccuracy and cycle counting on distribution center performance*, publicado en la revista *Production*, se propusieron analizar el impacto de la inexactitud de los registros de inventario (IRI) en el desempeño de los centros de distribución y determinar si el conteo cíclico permitía reducir sus efectos. Para ello, desarrollaron un modelo de dinámica de sistemas mediante el software Stella, con el que simularon durante un año los procesos de recepción, conteo cíclico y picking, considerando categorías de desempeño utilizadas por el WERC. El estudio abordó centros de distribución en los que los errores humanos durante la recepción y el picking generaban discrepancias entre las existencias físicas y las registradas. Los resultados mostraron que la IRI afectó negativamente los indicadores operativos y que el conteo cíclico permitió reducir sus efectos en determinadas categorías de desempeño, aunque no fue suficiente en las de menor rendimiento. Los autores concluyeron que era necesario supervisar la inexactitud de los registros junto con los indicadores de productividad. En relación con el presente proyecto, el estudio se concentró en la simulación del desempeño y en el control posterior de las discrepancias, sin describir un analizador formal que verificara la estructura léxica, sintáctica y semántica de cada movimiento antes de su aceptación.

Corella-Parra y Olea-Miranda (2023), en el artículo científico *Desarrollo de un sistema de control de inventario para una empresa comercializadora de sistemas de riego*, publicado en la revista

Ingeniería Investigación y Tecnología, tuvieron como objetivo desarrollar una metodología que mejorara el flujo de materiales y la precisión de los registros de inventario. La metodología comprendió el diagnóstico y modelado de procesos, la clasificación ABCD de productos según sus ventas anuales, la aplicación de controles internos, la gestión visual, el uso de códigos de barras y la incorporación de un sistema computarizado de inventario. El estudio se realizó en una empresa comercializadora de sistemas de riego de Sonora, México, que administraba más de mil tipos de productos y atendía aproximadamente cuarenta pedidos diarios. Entre los problemas identificados se encontraron registros conflictivos y un repositorio digital desactualizado. Los resultados reportaron un incremento de la satisfacción del cliente del 81 % al 93 % y una reducción del tiempo promedio de preparación de pedidos de 385,2 a 213,6, según los valores consignados en el antecedente. Los autores concluyeron que la integración de controles, organización y automatización contribuyó a mejorar la eficiencia operativa y a disminuir el riesgo de errores humanos en tareas manuales. Sin embargo, el trabajo no describió un lenguaje formal ni un mecanismo de análisis léxico, sintáctico y semántico para validar preventivamente las instrucciones de movimiento de inventario.

Jara-Saenz et al. (2026), en el artículo de conferencia *Data-driven inventory management and lean logistics integration: A case study in the Peruvian household cleaning products sector*, publicado en *AIP Conference Proceedings*, se propusieron optimizar la gestión de inventarios de una línea de detergentes perteneciente a una empresa multinacional del sector de productos de limpieza en el Perú. Para ello, combinaron técnicas de pronóstico mediante suavizado exponencial simple, el cálculo de la cantidad económica de pedido, la interrogación sistemática y la simulación de procesos mediante el software Arena, con treinta réplicas. El contexto del estudio estuvo relacionado con niveles de inventario superiores al rango considerado óptimo para la operación. Los resultados reportados mostraron una reducción de los días de inventario, una mejora de la rotación y una disminución de los costos de almacenamiento, aunque no se alcanzó completamente la meta establecida por los investigadores. Los autores concluyeron que la integración del pronóstico de demanda, las políticas de pedido y la logística esbelta contribuyó a mejorar la eficiencia de la gestión. Desde la perspectiva del presente proyecto, el antecedente abordó la planificación y optimización de existencias, pero no

desarrolló un analizador formal destinado a reconocer y validar la estructura y coherencia semántica de las instrucciones de entrada, salida, traslado y ajuste.

García Peña et al. (2023), en el artículo científico *Control interno de inventario y la gestión de resultados de un emporio comercial de la región de San Martín, Perú*, publicado en la revista *Región Científica*, tuvieron como objetivo determinar la relación entre el control interno de inventario y la gestión de resultados en negocios comerciales. Desarrollaron una investigación cuantitativa, no experimental, correlacional y transversal, aplicada a treinta negocios mayoristas de abarrotes. Para recopilar información utilizaron cuestionarios con escala Likert y el coeficiente de correlación de Spearman. El estudio se desarrolló en un contexto en el que se presentaban dificultades en la recepción de mercadería, el registro de productos y la identificación de fechas de vencimiento. Los resultados evidenciaron una correlación positiva y significativa entre las variables, con un coeficiente global de 0,740. Los autores concluyeron que un mayor nivel de control interno de inventarios se relacionaba con una mejor gestión de resultados. En contraste con el alcance de esta investigación, el estudio examinó la relación entre prácticas de control y gestión, pero no presentó el desarrollo de un sistema de análisis formal que verificara automáticamente las instrucciones de movimientos de inventario antes de su aceptación.

Vasquez-Layza et al. (2024), en el artículo de conferencia *Implementation of inventory control and warehouse management strategies to reduce operational costs in a Peruvian hardware company*, presentado en el evento LEIRD 2024 de LACCEI, tuvieron como objetivo implementar estrategias de control de inventarios y gestión de almacenes para reducir los costos operacionales de una empresa ferretera de Trujillo, Perú. Aplicaron un diseño preexperimental con mediciones previas y posteriores, mediante herramientas como un Kardex en Excel, clasificación ABC, reorganización de la distribución física del almacén y un plan de capacitación. Para contrastar las mediciones utilizaron la prueba t de Student. El contexto inicial presentó una exactitud de inventario del 20 %, un índice de rotura de stock de 43,63 % y registros digitales desactualizados. Los resultados evidenciaron una reducción de los costos operacionales, con una disminución promedio reportada de 14,149 % y una reducción máxima de 26,3 %. Los autores concluyeron que la implementación conjunta de las

estrategias permitió reducir significativamente los costos operacionales. No obstante, el estudio se concentró en herramientas de control y organización del inventario, sin describir un lenguaje específico de dominio ni un analizador formal que detectara errores léxicos, sintácticos y semánticos en las instrucciones de movimiento.

Floriano Rodríguez et al. (2024), en el artículo científico *Correlational analysis of internal control and inventory management in commercial SMEs of Chimbote*, publicado en la revista *Edelweiss Applied Science and Technology*, se propusieron determinar la relación entre el control interno y la gestión de inventarios en las micro y pequeñas empresas comerciales de Chimbote, Perú. La investigación siguió un enfoque cuantitativo y un diseño correlacional, y consideró una muestra de 160 MYPE. Para recopilar los datos se utilizaron cuestionarios con escala Likert y el coeficiente de correlación de Spearman. El contexto estuvo caracterizado por empresas que, en su mayoría, no contaban con un sistema formal de control interno. Los resultados mostraron una correlación positiva moderada entre las variables, con un coeficiente de 0,623, y señalaron que solo el 37,5 % de las empresas estudiadas gestionaba sus inventarios de manera eficiente. También se identificaron debilidades relacionadas con el pronóstico de la demanda y las políticas de inventario. Los autores concluyeron que el fortalecimiento del control interno y la optimización de la gestión de inventarios constituían aspectos importantes para la competitividad de estas empresas. En relación con el presente proyecto, el estudio examinó prácticas de control y su asociación con la gestión, pero no abordó la validación formal de instrucciones de inventario mediante análisis léxico, sintáctico y semántico.

Con el propósito de comparar los enfoques de las investigaciones revisadas, se sintetizaron sus contextos organizacionales, áreas o procesos estudiados, herramientas tecnológicas utilizadas, funciones abordadas y aspectos que no fueron cubiertos en relación con el alcance del presente proyecto. Esta comparación permitió distinguir las soluciones orientadas a la gestión operativa del inventario de la propuesta de validación formal preventiva de instrucciones de movimiento.

Tabla 1

Comparación de los antecedentes sobre gestión y control de inventarios

Artículo y año	Organización o contexto	Área o proceso	Software desarrollado o utilizado	Función principal	Vacío respecto del presente proyecto
Destro et al. (2023). The impacts of inventory record inaccuracy and cycle counting on distribution center performance.	Centros de distribución.	Recepción, conteo cíclico y picking.	Stella, utilizado para modelar y simular el comportamiento del sistema.	Analizar los efectos de la inexactitud de los registros y el alcance del conteo cíclico.	No se describe un analizador que valide formalmente cada instrucción de movimiento antes de aceptarla.
Corella-Parra y Olea-Miranda (2023). Desarrollo de un sistema de control de inventario para una empresa comercializadora de sistemas de riego.	Comercializadora de sistemas de riego de Sonora, México.	Control de inventario, flujo de materiales y preparación de pedidos.	Sistema computarizado de inventario y herramientas de código de barras; el artículo no identifica un lenguaje formal de validación.	Mejorar la precisión de los registros y el flujo de materiales mediante controles y automatización.	No se describe una gramática ni análisis léxico, sintáctico y semántico de instrucciones de inventario.
Jara-Saenz et al. (2026). Data-driven inventory management and lean logistics integration: A case study in the Peruvian household cleaning products sector.	Empresa multinacional del sector de productos de limpieza en el Perú.	Planificación de inventario y logística de una línea de detergentes.	Arena, utilizado para simulación; se aplicaron técnicas de pronóstico y cantidad económica de pedido.	Optimizar niveles de inventario, rotación y costos de almacenamiento.	No se desarrolla un procesador DSL para validar la estructura y el significado de movimientos individuales.

García Peña et al. (2023). Control interno de inventario y la gestión de resultados de un emporio comercial de la región de San Martín, Perú.	Treinta negocios mayoristas de abarrotes de la región San Martín.	Control interno y gestión de inventarios.	No especificado en el artículo revisado.	Determinar la relación entre el control interno de inventario y la gestión de resultados.	No se presenta un sistema informático de validación formal de movimientos de inventario.
Vasquez-Layza et al. (2024). Implementation of inventory control and warehouse management strategies to reduce operational costs in a Peruvian hardware company.	Empresa ferretera de Trujillo, Perú.	Control de inventario y gestión de almacén.	Kardex en Excel como herramienta de registro y control.	Aplicar estrategias de control y organización para reducir costos operacionales.	No se describe un lenguaje formal ni un analizador preventivo de instrucciones de inventario.
Floriano Rodríguez et al. (2024). Correlational analysis of internal control and inventory management in commercial SMEs of Chimbote.	160 MYPE comerciales de Chimbote, Perú.	Control interno y gestión de inventarios.	No especificado en el artículo revisado.	Analizar la relación entre el control interno y la gestión de inventarios.	No se desarrolla un mecanismo computacional que analice léxica, sintáctica y semánticamente los movimientos.

Nota. Elaboración propia a partir de los seis antecedentes revisados. La columna «Vacío respecto del presente proyecto» expresa una comparación del alcance de los estudios con la propuesta de validación formal; no representa necesariamente una limitación declarada por sus autores. Cuando el artículo no identifica una herramienta informática, se consigna «No especificado en el artículo revisado».

En conjunto, los seis antecedentes abordaron dificultades relacionadas con la exactitud de los registros, el control de existencias y el desempeño de los procesos de inventario en organizaciones

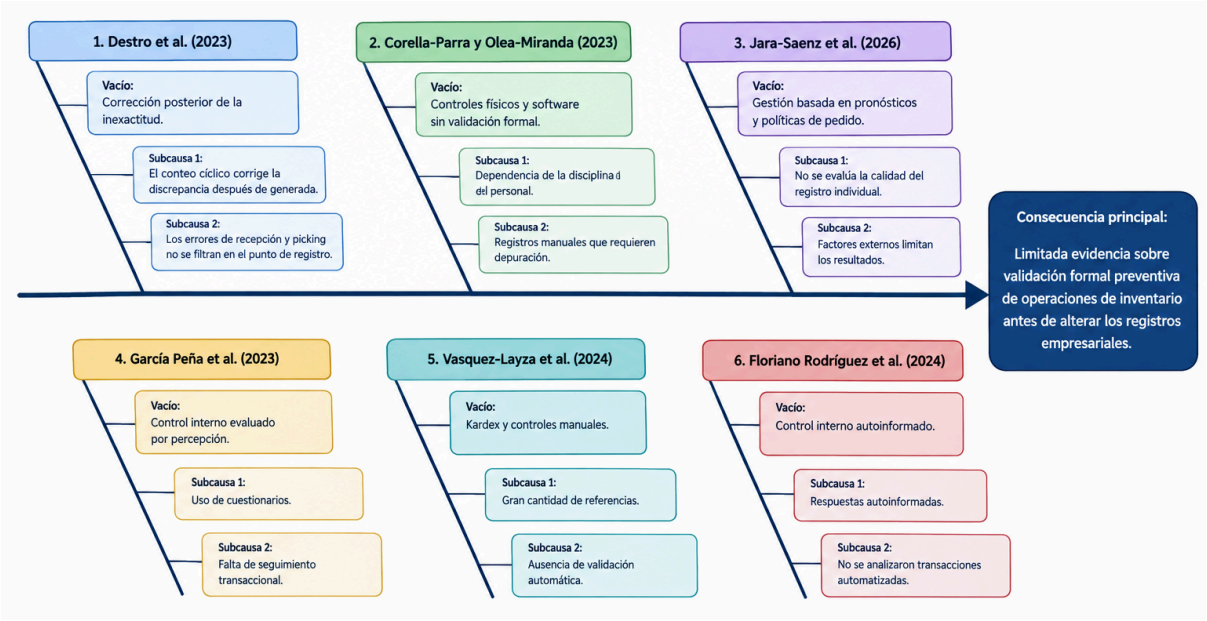
comerciales y centros de distribución. Aunque emplearon enfoques diferentes, coincidieron en la necesidad de reducir discrepancias entre las existencias físicas y la información registrada, así como de fortalecer las actividades de recepción, almacenamiento, reposición y despacho. Para ello, recurrieron a estrategias como el conteo cíclico, la clasificación ABC, los sistemas computarizados, el uso de Kardex y códigos de barras, la planificación de la demanda, la simulación y el control interno. Sin embargo, los estudios revisados se concentraron principalmente en la gestión de las existencias, la organización de los procesos o la identificación de relaciones entre variables; no describieron una solución que tratara cada movimiento de inventario como una instrucción perteneciente a un lenguaje formal y que verificara preventivamente su estructura y coherencia antes de aceptarlo. Por consiguiente, el presente proyecto se diferenció al proponer un analizador DSL especializado que integró análisis léxico, sintáctico y semántico para examinar instrucciones de entrada, salida, traslado y ajuste en un caso de estudio hipotético de una empresa comercial de Lima.

A partir de la comparación de los antecedentes, se identificó como vacío general la falta de evaluación de un mecanismo de validación formal que analizara cada movimiento de inventario antes de su aceptación, mediante reglas léxicas, sintácticas y semánticas integradas. Las investigaciones revisadas abordaron la inexactitud de los registros mediante controles operativos, herramientas de gestión, sistemas de inventario, simulaciones o análisis del control interno; sin embargo, no describieron un procedimiento que reconociera las instrucciones de movimiento como expresiones de un lenguaje específico de dominio, comprobara su estructura gramatical y verificara la coherencia de sus datos con las condiciones del inventario. Este vacío se relacionó con el problema que motivó el proyecto: la posibilidad de que instrucciones con formatos incorrectos, campos incompletos, identificadores no reconocidos o cantidades incompatibles fueran aceptadas sin una revisión formal previa. En consecuencia, se planteó implementar un analizador DSL especializado que procesara movimientos de entrada, salida, traslado y ajuste, y que emitiera diagnósticos cuando las instrucciones no cumplieran las reglas establecidas. La propuesta se examinó mediante un caso hipotético y datos simulados, sin atribuir sus resultados a una aplicación empresarial real.

La comparación de los seis antecedentes permitió organizar los aspectos no abordados por las investigaciones en seis grupos de causas, uno por cada estudio revisado. Estos grupos representaron diferencias de alcance entre las soluciones estudiadas y la validación formal propuesta. El efecto común identificado correspondió a la ausencia de una evaluación de un mecanismo que verificara léxica, sintáctica y semánticamente cada movimiento de inventario antes de su aceptación.

Figura 1

Diagrama de Ishikawa del vacío de conocimiento identificado en los antecedentes



Nota. Elaboración propia a partir de los seis antecedentes revisados. Las ramas representan aspectos no abordados dentro del alcance de los estudios analizados y no limitaciones atribuidas expresamente a sus autores.

Problemática

La gestión de inventarios requiere mantener correspondencia entre las existencias físicas y los registros que representan las entradas, salidas, traslados y ajustes de productos. En el caso hipotético considerado, los movimientos podían registrarse mediante instrucciones textuales sin una validación formal previa, lo que permitía que errores de formato, estructuras incorrectas o datos incompatibles con las existencias llegaran a las etapas posteriores del proceso. Esta situación podía ocasionar

diferencias entre el inventario registrado y el disponible, dificultades de trazabilidad y decisiones operativas basadas en información inexacta. Los antecedentes evidenciaron la relevancia del problema: García Peña et al. (2023) reportaron una correlación de 0,740 entre el control interno y la gestión de resultados; Vasquez-Layza et al. (2024) identificaron una exactitud inicial del inventario del 20 % y un índice de quiebres de stock del 43,63 % en el caso estudiado; mientras que Floriano Rodríguez et al. (2024) encontraron que el 37,5 % de las empresas analizadas presentaba una gestión de inventarios eficiente. Sin embargo, estos trabajos no abordaron la validación formal de instrucciones mediante un lenguaje específico del dominio. Por ello, se planteó estudiar la relación entre la validación léxica, sintáctica y semántica de movimientos de inventario, como variable independiente, y la detección de inconsistencias en dichos movimientos, como variable dependiente, mediante un prototipo evaluado con información simulada.

Justificación

El proyecto se justificó, en primer lugar, desde una perspectiva teórica, porque permitió aplicar conceptos de lenguajes formales y teoría de compiladores a un problema relacionado con la gestión de inventarios. La representación de entradas, salidas, traslados y ajustes mediante un lenguaje específico del dominio permitió establecer reglas explícitas para reconocer los componentes de cada instrucción, comprobar su estructura y evaluar su significado frente a las restricciones del inventario. De esta manera, se vinculó el reconocimiento léxico, el análisis sintáctico y la validación semántica con un proceso operativo concreto.

Desde la perspectiva práctica, se propuso una validación preventiva de las instrucciones antes de aceptar un lote de movimientos. El prototipo permitió examinar si los datos declarados correspondían a productos, ubicaciones, referencias y motivos registrados, así como comprobar las condiciones relacionadas con las cantidades y las existencias disponibles. Esta propuesta buscó identificar inconsistencias antes de que una operación pudiera incorporarse a un registro empresarial. Sin embargo, la solución no sustituyó un sistema de gestión de inventarios ni ejecutó una actualización

definitiva de las existencias. El archivo data/stock.json se utilizó como fuente del inventario inicial y no fue modificado durante el procesamiento.

En el ámbito metodológico, el trabajo organizó la evaluación del analizador mediante casos de prueba destinados a representar instrucciones válidas y situaciones de error léxico, sintáctico y semántico. Esta estructuración permitió comprobar el comportamiento del sistema ante diferentes condiciones de entrada y conservar evidencias de los resultados obtenidos. La evaluación se planteó como una comprobación funcional del prototipo, no como una medición de mejoras operativas en una empresa real.

Desde la perspectiva tecnológica, se utilizó ANTLR 4 para generar el analizador léxico y sintáctico a partir de la gramática Inventario.g4. Python integró el procesamiento de las instrucciones, la transformación de los movimientos mediante el componente visitante y la aplicación de las reglas semánticas. Asimismo, Flask proporcionó una interfaz web para ingresar o cargar documentos DSL y presentar los resultados del análisis. La solución se publicó como una demostración accesible mediante un entorno web. Tanto el escenario empresarial como los datos utilizados fueron hipotéticos y simulados; por consiguiente, la publicación permitió demostrar el funcionamiento del prototipo, pero no acreditar su integración con una organización o un sistema empresarial existente.

Áreas de la empresa

Para delimitar el escenario empresarial utilizado en el proyecto, se consideraron las áreas funcionales que participan directamente en el flujo de los movimientos de inventario. Debido a que el caso de estudio es hipotético, estas áreas representan una estructura funcional de referencia y no corresponden a una organización real intervenida. Su inclusión permite identificar el origen de las operaciones, las responsabilidades relacionadas con el registro de existencias y los puntos en los que la información del inventario resulta necesaria.

El área de Compras representa la función encargada de gestionar el abastecimiento y coordinar la adquisición de productos. Su participación se relaciona principalmente con las entradas de mercadería,

debido a que las órdenes de compra y los documentos de recepción pueden servir como referencias para respaldar el ingreso de productos al inventario. En el escenario propuesto, esta área proporciona información que permite contextualizar el movimiento, pero no realiza directamente el análisis del DSL.

El área de Almacén e inventarios constituye el núcleo operativo del caso de estudio. Comprende la recepción, ubicación, conservación, traslado y despacho de los productos, así como el registro de ajustes cuando se identifican diferencias justificadas. Sus actividades se relacionan con los cuatro movimientos representados por el lenguaje: ENTRADA, SALIDA, TRASLADO y AJUSTE. La información sobre productos, ubicaciones y existencias constituye la base para las comprobaciones semánticas del analizador.

El área de Ventas interviene cuando se generan requerimientos de productos para atender pedidos. Su relación con el inventario se manifiesta en la consulta de disponibilidad y en la información que respalda las salidas por despacho. En el escenario planteado, la operación de venta no se representa como una instrucción DSL independiente; únicamente se considera el movimiento de inventario asociado con la entrega de los productos.

El área de Administración o Contabilidad participa en el control documental, la revisión de registros y la conciliación de las existencias físicas con las cantidades registradas. Su intervención permite contextualizar la importancia de conservar referencias y datos coherentes en los movimientos. No obstante, la valorización contable, la facturación y los cierres financieros no forman parte de las operaciones procesadas por el DSL.

Finalmente, la Tecnología de la Información se considera únicamente como una función de soporte técnico para el prototipo. Su mención permite situar el desarrollo y mantenimiento de la demostración informática, pero no implica que se haya intervenido un departamento tecnológico de una empresa real. El analizador se diseñó como una solución independiente que podría servir como componente de validación en una futura integración, siempre que se desarrollaran los mecanismos correspondientes.

La Tabla 2 resume las responsabilidades consideradas para cada área y su relación con los movimientos de inventario y la solución propuesta. Esta clasificación permite distinguir las funciones operativas y administrativas del escenario hipotético respecto del soporte tecnológico requerido para el prototipo.

Tabla 2

Áreas consideradas y su participación en el proceso de inventarios

Área	**Responsabilidad en el caso de estudio**	**Movimientos relacionados**	**Relación con la solución propuesta**
Compras	Gestionar el abastecimiento y proporcionar información documental de las adquisiciones.	ENTRADA	La documentación de compra puede respaldar la referencia declarada en el movimiento.
Almacén e inventarios	Recibir, ubicar, conservar, trasladar y despachar productos; registrar ajustes de existencias.	ENTRADA, SALIDA, TRASLADO y AJUSTE	Constituye el ámbito operativo principal cuyos movimientos son representados y validados mediante el DSL.
Ventas	Gestionar los pedidos y proporcionar información relacionada con los productos solicitados.	SALIDA	La disponibilidad de productos y la referencia del despacho contextualizan la salida de inventario.
Administración o Contabilidad	Revisar documentación y conciliar las existencias físicas con los registros.	ENTRADA, SALIDA, TRASLADO y AJUSTE	Permite contextualizar la necesidad de mantener registros coherentes y trazables.

Tecnología de la Información	Proporcionar soporte técnico para el desarrollo y funcionamiento del prototipo.	No ejecuta movimientos empresariales	Se relaciona con la implementación tecnológica de la demostración, no con una intervención organizacional real.

Nota. Elaboración propia a partir de las funciones consideradas para el caso de estudio hipotético. Tecnología de la Información se incluye como función de soporte del prototipo y no como un área empresarial intervenida.

Procesos principales

Los procesos principales considerados en el caso de estudio corresponden a las actividades necesarias para mantener el control de los productos desde su recepción hasta su salida, así como para conservar la correspondencia entre las existencias físicas y los registros de inventario. Estos procesos se relacionan con las operaciones que el DSL permite representar, aunque no abarcan la totalidad de la gestión administrativa de la organización hipotética.

El proceso de abastecimiento y recepción de productos comprende la identificación de los bienes adquiridos, la revisión de las cantidades recibidas y la comprobación de la documentación que respalda el ingreso. Desde la perspectiva del inventario, esta actividad se representa mediante un movimiento ENTRADA, que incorpora una cantidad determinada de un producto a una ubicación de destino.

El proceso de almacenamiento y ubicación comprende la asignación de espacios para conservar los productos y la identificación de las ubicaciones en las que se encuentran las existencias. Cuando un producto se cambia de una ubicación a otra, el movimiento se representa mediante TRASLADO, que registra tanto el origen como el destino. La operación modifica la distribución de las existencias entre ubicaciones, sin que ello implique por sí mismo un cambio en la cantidad global del producto.

El proceso de salida y despacho comprende la atención de los requerimientos de productos, la consulta de disponibilidad, la preparación de los bienes y su retiro del almacén. La disminución de las

existencias se representa mediante SALIDA, que identifica el producto, la cantidad, la ubicación de origen y la referencia correspondiente.

El proceso de ajuste de inventario se considera cuando se requiere corregir una diferencia justificada entre las existencias registradas y las cantidades verificadas. El movimiento AJUSTE permite representar un incremento o decremento asociado con un motivo y una referencia, de acuerdo con las reglas definidas para el lenguaje.

Por último, el proceso de control y conciliación de existencias comprende la revisión de los registros y su contraste con la información disponible sobre el inventario físico. Este proceso permite contextualizar la necesidad de mantener movimientos consistentes y respaldados. Sin embargo, la conciliación contable y administrativa no se ejecuta como una operación independiente del DSL, cuyo alcance se limita a representar y validar movimientos de inventario.

En consecuencia, el lenguaje específico del dominio constituye un mecanismo de representación y validación de operaciones de inventario, pero no reemplaza los procesos administrativos de compras, ventas, contabilidad o gestión de proveedores. Su función se concentra en comprobar que las instrucciones que describen los movimientos cumplan las condiciones léxicas, sintácticas y semánticas establecidas antes de presentar su resultado.

La Tabla 3 presenta la correspondencia entre los procesos principales del caso de estudio y los movimientos representados por el DSL. También resume los datos que intervienen en cada operación y el resultado esperado desde la perspectiva del control de existencias.

Tabla 3

Procesos principales relacionados con la gestión de inventarios

Proceso	**Descripción**	**Movimiento DSL relacionado**	**Datos principales**	**Resultado esperado**

Abastecimiento y recepción	Verificación de los productos y cantidades recibidas, junto con la documentación correspondiente.	ENTRADA	SKU, cantidad, ubicación de destino y referencia.	Incorporación de existencias a la ubicación indicada, si la instrucción es válida.
Almacenamiento y ubicación	Organización de productos y cambios de ubicación dentro del almacén.	TRASLADO	SKU, cantidad, ubicación de origen, ubicación de destino y referencia.	Disminución del saldo en el origen e incremento en el destino, sin alterar el total global.
Salida y despacho	Preparación y retiro de productos para atender un requerimiento.	SALIDA	SKU, cantidad, ubicación de origen y referencia.	Disminución de existencias cuando existe disponibilidad suficiente.
Ajuste de inventario	Corrección justificada de las cantidades registradas.	AJUSTE	SKU, cantidad, tipo de ajuste, ubicación, motivo y referencia.	Incremento o decremento del saldo conforme a las reglas semánticas.
Control y conciliación	Revisión de la correspondencia entre las existencias físicas y los registros.	Se relaciona con los cuatro movimientos; no constituye un movimiento DSL independiente.	Registros de movimientos, productos, ubicaciones y existencias.	Identificación de diferencias que requieren revisión y respaldo documental.

Nota. Elaboración propia. Los datos principales corresponden a los campos contemplados por el DSL. El resultado esperado describe el efecto conceptual de cada movimiento y no representa una actualización definitiva del inventario empresarial.

Subprocesos

Los subprocesos del caso de estudio se organizaron en dos grupos: los relacionados con la operación empresarial y los correspondientes al funcionamiento de la solución propuesta. Esta separación

permite distinguir las actividades que originan y preparan los movimientos de inventario de las etapas técnicas que intervienen en la recepción, el análisis y la validación de las instrucciones DSL.

En el ámbito empresarial, el subproceso de registro de una entrada comprende la identificación del producto recibido, la determinación de la cantidad, la asignación de una ubicación de destino y la identificación de la referencia que respalda la operación. El subproceso de registro de una salida comprende la identificación del producto solicitado, la consulta de disponibilidad, la determinación de la cantidad que se retirará y la identificación de la ubicación de origen. En ambos casos, la información preparada se utiliza para formular el movimiento correspondiente.

El subproceso de registro de un traslado comprende la identificación del producto, la cantidad que se desplazará y las ubicaciones de origen y destino. Por su parte, el subproceso de registro de un ajuste comprende la identificación de la diferencia que se pretende corregir, la selección del tipo de ajuste, la determinación de la ubicación afectada y la consignación del motivo y de la referencia correspondiente.

La preparación de estos movimientos requiere comprobar la información que los respalda. Por ello, se consideran actividades de consulta y verificación de productos, ubicaciones, referencias y motivos autorizados. También se contempla la consulta de disponibilidad para las operaciones que disminuyen existencias. Estas comprobaciones permiten determinar si los datos declarados son compatibles con la información de referencia y con las restricciones del inventario.

En el ámbito de la solución, el proceso comienza con la recepción del documento DSL, ya sea mediante el ingreso de texto o la carga de un archivo admitido. A continuación, el analizador realiza el reconocimiento léxico de los componentes y verifica la estructura de las instrucciones mediante el análisis sintáctico. Cuando la estructura es válida, el componente VisitorMovimientos transforma los movimientos reconocidos en estructuras que pueden ser procesadas por la validación semántica.

La validación semántica comprueba las reglas M01 a M09 con apoyo de los datos de referencia y del inventario inicial. Para evaluar un lote con varios movimientos, se utiliza una copia temporal de las

existencias y se procesan las operaciones en el orden declarado. Así, cada movimiento posterior considera el saldo proyectado por los movimientos anteriores del mismo lote. Esta evaluación no modifica el archivo data/stock.json.

Finalmente, el sistema determina si el lote puede aceptarse o debe rechazarse. Cuando se identifican errores, se presentan los diagnósticos correspondientes; cuando las comprobaciones se satisfacen, se construye el resultado JSON de aceptación y se muestra el stock proyectado. En ambos casos, Flask presenta el resultado del análisis en la interfaz web.

La Tabla 4 organiza los subprocesos según su relación con el escenario empresarial o con el funcionamiento del prototipo. La clasificación permite identificar qué actividades preparan los movimientos y cuáles corresponden exclusivamente al procesamiento del DSL.

Tabla 4

Subprocesos empresariales y subprocesos de la solución propuesta

Grupo	Subproceso	Descripción
Empresarial	Registrar una entrada	Identificar el producto recibido, la cantidad, la ubicación de destino y la referencia de respaldo.
Empresarial	Registrar una salida	Identificar el producto, consultar su disponibilidad, determinar la cantidad y establecer la ubicación de origen.
Empresarial	Registrar un traslado	Identificar el producto, la cantidad y las ubicaciones de origen y destino.
Empresarial	Registrar un ajuste	Determinar el tipo de ajuste, la ubicación afectada, el motivo y la referencia correspondiente.

Empresarial	Validar datos del producto	Comprobar que el SKU declarado corresponda a un producto incluido en el catálogo.
Empresarial	Validar ubicación	Comprobar que las ubicaciones declaradas estén registradas y sean aplicables al movimiento.
Empresarial	Validar referencia	Comprobar que la referencia exista y sea compatible con el tipo de movimiento.
Empresarial	Validar motivo	Comprobar que el motivo declarado esté autorizado para el ajuste correspondiente.
Empresarial	Consultar disponibilidad	Verificar que exista saldo suficiente para las operaciones que disminuyen las existencias.
Empresarial	Calcular el stock proyectado	Evaluar el efecto acumulado de los movimientos en el orden en que aparecen en el lote.
Solución	Recibir el documento DSL	Admitir el texto ingresado o el archivo proporcionado para su procesamiento.
Solución	Realizar el análisis léxico	Reconocer los tokens y detectar componentes que no cumplen las reglas léxicas.
Solución	Realizar el análisis sintáctico	Comprobar que los tokens formen instrucciones válidas según la gramática.
Solución	Transformar los movimientos	Utilizar `VisitorMovimientos` para obtener las estructuras de datos que serán evaluadas semánticamente.
Solución	Ejecutar la validación semántica	Aplicar las reglas M01 a M09 con los datos de referencia y el saldo temporal.

Solución	Aceptar o rechazar el lote	Determinar el resultado global según los errores identificados durante el procesamiento.
Solución	Presentar el resultado	Mostrar los diagnósticos o el JSON de aceptación y el stock proyectado mediante Flask.

Nota. Elaboración propia a partir de los subprocesos del caso hipotético y de la implementación del prototipo. Los componentes léxico, sintáctico, visitante y semántico son etapas internas de la solución, no áreas ni cargos de la organización.

Proceso general del negocio

El proceso general del negocio representa la secuencia mediante la cual se originan, preparan y validan los movimientos de inventario en el escenario hipotético. Su finalidad es mostrar la relación entre las actividades operativas de la organización y el punto en el que interviene el analizador DSL, sin confundir la validación de instrucciones con la ejecución completa de los procesos administrativos.

El proceso comienza cuando surge la necesidad de registrar una operación relacionada con las existencias. Esta necesidad puede originarse por la recepción de productos, el despacho de un pedido, el desplazamiento de mercadería entre ubicaciones o la identificación de una diferencia que requiera un ajuste. A partir de esta necesidad, se identifica el tipo de movimiento que corresponde: ENTRADA, SALIDA, TRASLADO o AJUSTE.

Una vez determinado el movimiento, se preparan los datos necesarios, como el código del producto, la cantidad, las ubicaciones que correspondan, la referencia documental y, en el caso de los ajustes, el tipo y el motivo. Con esta información se formula una instrucción en el lenguaje específico del dominio o se prepara un archivo que contiene uno o más movimientos agrupados en un lote.

El documento se ingresa en el prototipo y es recibido por el analizador DSL. Este ejecuta el análisis léxico, comprueba la estructura mediante el análisis sintáctico y transforma los movimientos reconocidos utilizando VisitorMovimientos. Posteriormente, se aplican las reglas semánticas M01 a

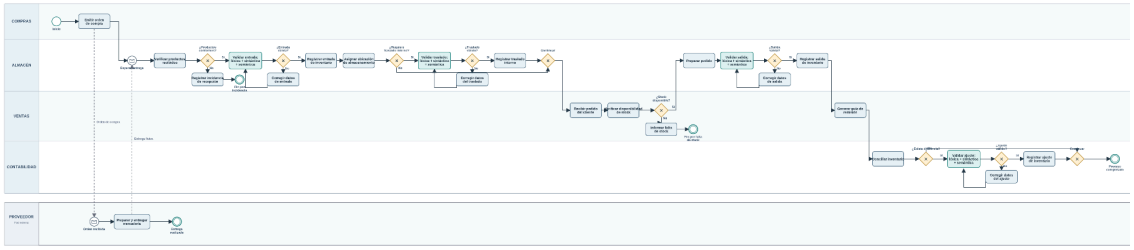
M09 con apoyo de los datos de referencia y de una copia temporal del inventario inicial. El procesamiento considera el orden de los movimientos declarados en el lote y no modifica el archivo original de existencias.

Como resultado de la evaluación, el lote se acepta cuando no presenta errores en las etapas de análisis y validación. En ese caso, el sistema construye el JSON de aceptación y presenta el stock proyectado. Si se identifican errores, el lote se rechaza y se muestran los diagnósticos correspondientes. En ambos escenarios, Flask presenta el resultado en la interfaz web. La figura representa este flujo y delimita la intervención del analizador dentro del proceso empresarial.

La Figura 2 representa el proceso de registro de movimientos de inventario y la intervención del analizador DSL antes de presentar el resultado de la evaluación. El flujo diferencia la preparación de las instrucciones, las etapas de análisis y validación, y las dos respuestas posibles del prototipo: aceptación o rechazo del lote.

Figura 2

Proceso general de movimientos de inventario y ubicación del analizador DSL



Nota. Elaboración propia a partir del proceso empresarial hipotético y del flujo funcional de la solución implementada.

Como se observa en la Figura 2, el analizador se ubica entre la preparación de la instrucción y la presentación del resultado, por lo que su función consiste en evaluar la validez de los movimientos antes de aceptarlos. La validación semántica consulta los datos de referencia y utiliza una copia temporal del stock para considerar el efecto acumulado de las operaciones. Si se detecta algún error, el lote se rechaza y se presentan los diagnósticos; si todas las comprobaciones se satisfacen, se genera el

resultado JSON de aceptación y se muestra el stock proyectado. En ninguno de los dos casos se modifica el archivo data/stock.json, por lo que el prototipo no ejecuta una actualización definitiva de las existencias.

Organigrama

Con la finalidad de contextualizar las áreas consideradas en el caso de estudio, se presenta una estructura organizacional hipotética que representa las principales funciones relacionadas con la gestión de inventarios. Esta estructura tiene carácter referencial y se utiliza exclusivamente para delimitar las responsabilidades consideradas en el proceso descrito. Por tanto, no corresponde al organigrama oficial de una empresa real ni representa una organización en la que se haya realizado una intervención.

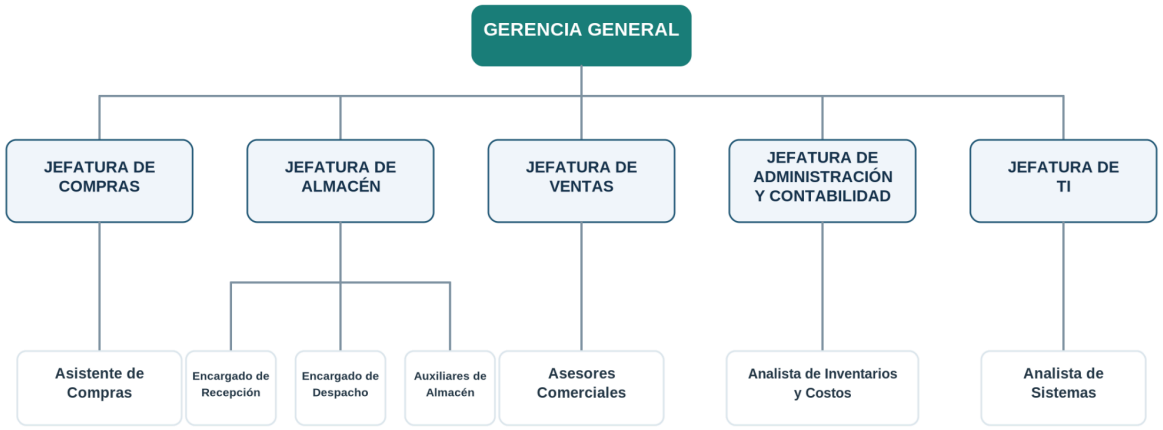
En el nivel superior se ubica la Gerencia General, como instancia de dirección y coordinación de las áreas consideradas. Bajo esta estructura se identifican Compras, Almacén e Inventarios, Ventas y Administración o Contabilidad. Compras se relaciona con el abastecimiento y la documentación que respalda las entradas; Almacén e Inventarios concentra las actividades de recepción, ubicación, traslado, despacho y control de existencias; Ventas se relaciona con los requerimientos que pueden originar salidas; y Administración o Contabilidad participa en la revisión documental y la conciliación de los registros.

El soporte tecnológico requerido para el desarrollo y funcionamiento del prototipo se considera una función técnica complementaria. No se representa como un área organizacional intervenida ni como parte de la estructura empresarial necesaria para ejecutar los movimientos de inventario. De esta manera, el organigrama mantiene su atención en las áreas que permiten comprender el proceso de negocio y evita incorporar cargos o unidades que no resultan indispensables para el alcance del proyecto.

La Figura 3 presenta la estructura funcional considerada para el escenario hipotético. Su propósito es ubicar las áreas que participan en las actividades relacionadas con el inventario, sin atribuir cargos, trabajadores o dependencias específicas a una empresa real.

Figura 3

Organigrama referencial de la organización considerada en el caso de estudio



Nota. Elaboración propia. La figura corresponde a una estructura organizacional hipotética utilizada para delimitar el caso de estudio.

OBJETIVOS

Objetivo general

Implementar una validación léxica, sintáctica y semántica de movimientos de inventario para detectar inconsistencias en un caso simulado de empresa comercial de Lima.

Objetivos específicos

1. Identificar los errores léxicos, sintácticos y semánticos que pueden generar inconsistencias en los movimientos de inventario del caso simulado.
2. Formalizar el DSL mediante alfabeto, expresiones regulares, autómatas finitos y gramática libre de contexto para los cuatro movimientos.

3. Generar el analizador léxico y sintáctico con ANTLR 4.13.2 a partir de Inventario.g4 para reconocer las instrucciones definidas.
4. Integrar la transformación de movimientos y las reglas semánticas M01–M09 con el procesamiento secuencial de lotes y la interfaz web.
5. Evaluar el analizador mediante casos de regresión y evidencias de aceptación, rechazo y conservación del archivo de existencias.

La Tabla 5 relaciona cada objetivo específico con la fase de implementación en la que se desarrolla principalmente y con las evidencias que permitirán comprobar su cumplimiento. Esta correspondencia facilita mantener la trazabilidad entre los propósitos del proyecto y los productos obtenidos.

Tabla 5

Correspondencia entre objetivos específicos, fases y evidencias de verificación

Objetivo específico	**Palabras**	**Fase Waterfall relacionada**	**Evidencia de verificación**
OE1. Identificar los errores léxicos, sintácticos y semánticos que pueden generar inconsistencias en los movimientos de inventario del caso simulado.	19	Análisis	Clasificación de errores y catálogo de condiciones léxicas, sintácticas y semánticas.
OE2. Formalizar el DSL mediante alfabeto, expresiones regulares, autómatas finitos y gramática libre de contexto para los cuatro movimientos.	18	Diseño	Especificación del lenguaje, expresiones regulares, autómatas y gramática documentada.

OE3. Generar el analizador léxico y sintáctico con ANTLR 4.13.2 a partir de Inventario.g4 para reconocer las instrucciones definidas.	18	Desarrollo	Archivo Inventario.g4, componentes generados por ANTLR y ejecución del analizador.
OE4. Integrar la transformación de movimientos y las reglas semánticas M01–M09 con el procesamiento secuencial de lotes y la interfaz web.	20	Desarrollo	Implementación de VisitorMovimientos, validador semántico, procesamiento de lotes e interfaz Flask.
OE5. Evaluar el analizador mediante casos de regresión y evidencias de aceptación, rechazo y conservación del archivo de existencias.	18	Pruebas	Matriz de casos de regresión, registros de ejecución, diagnósticos y comprobación de que data/stock.json no fue modificado.

Nota. Elaboración propia. Las fases indican la relación principal de cada objetivo con la metodología Waterfall; algunas actividades pueden requerir verificaciones complementarias durante fases posteriores.

MARCO TEÓRICO

El Marco teórico establece los fundamentos conceptuales que sustentan la propuesta de validación formal de movimientos de inventario. Para ello, relaciona los principios de la gestión de inventarios con los conceptos de la teoría de lenguajes y de los compiladores, debido a que el problema abordado comprende tanto la consistencia de la información sobre las existencias como la estructura y el significado de las instrucciones utilizadas para representar sus movimientos. Los cinco conceptos desarrollados permiten comprender el dominio empresarial, la formalización de un lenguaje, el reconocimiento de sus componentes, la comprobación de su estructura y la evaluación de las restricciones propias del inventario. Esta sección presenta los fundamentos teóricos que justifican el enfoque adoptado, mientras que la especificación concreta del lenguaje y su implementación se desarrollan posteriormente.

1. Gestión de inventarios e inexactitud de registros

La gestión de inventarios comprende las actividades destinadas a controlar el flujo de materiales y mantener las existencias en niveles que permitan atender las necesidades de la organización. En este proceso se consideran operaciones como la recepción, el almacenamiento, la ubicación, la preparación de pedidos y el despacho de productos. Su adecuada administración requiere que la información registrada represente de manera confiable las cantidades y ubicaciones de los artículos disponibles, puesto que las decisiones de reposición y atención de pedidos dependen de esos datos. Corella-Parra y Olea-Miranda (2023) relacionaron el control de inventarios con el seguimiento del flujo de materiales y la conservación de niveles de producto apropiados para responder a la demanda.

La correspondencia entre las existencias físicas y los registros constituye una condición importante para la confiabilidad del inventario. Destro et al. (2023), a partir de la definición de DeHoratius y Raman (2008), explicaron la inexactitud de los registros de inventario (Inventory Record Inaccuracy, IRI) como la discrepancia entre la cantidad que figura en el sistema y la cantidad existente físicamente. Esta diferencia puede manifestarse como inventario fantasma, cuando el registro indica una cantidad superior a la disponible, o como inventario oculto, cuando la existencia física supera la cantidad registrada. Estas situaciones pueden originarse por errores durante la recepción, el almacenamiento, la preparación de pedidos o el despacho, así como por registros incompletos o desactualizados.

La inexactitud puede afectar distintas actividades operativas. Durante la recepción, una cantidad registrada incorrectamente puede generar diferencias entre los productos recibidos y las existencias del sistema. En el almacenamiento, una ubicación equivocada dificulta localizar los artículos. Durante el picking y el despacho, la información inexacta puede provocar búsquedas innecesarias, preparación incompleta de pedidos o ventas que no pueden atenderse con las existencias disponibles. Destro et al. (2023) señalaron que la IRI puede perjudicar la productividad del picking, las ventas y la utilización de la capacidad del centro de distribución. Por ello, el control de inventarios no depende únicamente de conservar registros, sino también de procurar que estos reflejen adecuadamente las operaciones realizadas.

Los movimientos de entrada, salida, traslado y ajuste representan modificaciones sobre las existencias registradas. Una entrada incrementa la cantidad disponible; una salida la reduce; un traslado cambia la ubicación asociada al producto; y un ajuste registra una variación justificada de la cantidad existente. Si la información que representa estas operaciones contiene códigos incorrectos, cantidades no permitidas, ubicaciones inexistentes o referencias incompatibles, el registro resultante puede perder consistencia. Por esa razón, además de los controles físicos y administrativos, resulta pertinente considerar mecanismos preventivos que examinen la estructura y la coherencia de las instrucciones antes de aceptarlas. Esta validación no sustituye los conteos físicos ni garantiza por sí sola que el inventario registrado coincida con la realidad, pero permite controlar determinadas inconsistencias presentes en la representación de los movimientos.

2. Lenguajes formales, expresiones regulares y autómatas finitos

Un lenguaje formal es un conjunto de cadenas construidas a partir de los símbolos de un alfabeto, de acuerdo con determinadas condiciones de formación. Hopcroft, Motwani y Ullman (2007) desarrollaron estos conceptos como fundamentos de la teoría de autómatas y lenguajes formales. Un alfabeto es un conjunto finito y no vacío de símbolos; cada elemento del conjunto se denomina símbolo. Una cadena es una secuencia finita de símbolos pertenecientes a un alfabeto, mientras que un lenguaje es un conjunto de cadenas definidas sobre ese alfabeto. Estos conceptos permiten establecer de manera matemática qué secuencias pertenecen a un lenguaje y cuáles quedan fuera de él.

Las expresiones regulares constituyen una notación formal para describir lenguajes regulares. Se construyen mediante operaciones como la unión, la concatenación y la repetición, y permiten especificar patrones que deben cumplir determinadas cadenas. En el procesamiento de lenguajes, se utilizan para describir la forma de elementos individuales, como identificadores, números, palabras reservadas o delimitadores. Su utilidad consiste en expresar de manera compacta los patrones que posteriormente pueden ser reconocidos por mecanismos computacionales.

Los autómatas finitos son modelos matemáticos que reconocen lenguajes regulares mediante estados y transiciones. Un autómata finito no determinista (AFND) puede admitir varias transiciones posibles

para un mismo símbolo desde un estado, así como transiciones vacías cuando la definición del modelo las contempla. Un autómata finito determinista (AFD), en cambio, establece una única transición para cada combinación de estado y símbolo de entrada, de acuerdo con su función de transición. Ambos modelos poseen la misma capacidad de reconocimiento: para todo AFND existe un AFD equivalente que reconoce el mismo lenguaje (Hopcroft et al., 2007). Esta equivalencia permite transformar una representación no determinista en otra determinista sin modificar el conjunto de cadenas reconocidas.

La relación entre expresiones regulares y autómatas finitos resulta especialmente importante en el análisis léxico. Los patrones descritos mediante expresiones regulares pueden reconocerse mediante autómatas, los cuales permiten determinar si una secuencia de caracteres corresponde a una categoría válida. De esta manera, los fundamentos de los lenguajes formales proporcionan herramientas para especificar y reconocer los componentes elementales de un lenguaje.

En el proyecto, estos conceptos sustentan la formalización de las instrucciones que representan movimientos de inventario. Su aplicación concreta comprende la definición de los símbolos admitidos y de los patrones necesarios para reconocer los componentes de las instrucciones, aspectos que se desarrollan en la fase de Análisis.

3. Lenguaje específico de dominio

Un lenguaje específico de dominio (Domain-Specific Language, DSL) es un lenguaje diseñado para expresar conceptos, operaciones y restricciones de un ámbito particular. A diferencia de un lenguaje de propósito general, que busca ofrecer mecanismos de programación aplicables a una variedad amplia de problemas, un DSL utiliza una representación especializada para facilitar la descripción de actividades propias de un dominio. Su vocabulario y sus construcciones se organizan en función de las necesidades del ámbito en el que será utilizado.

La principal característica de un DSL es la correspondencia entre sus expresiones y los conceptos del dominio. Esta correspondencia permite representar operaciones mediante instrucciones que resultan más cercanas al problema que se desea resolver. En lugar de utilizar estructuras generales de

programación para cada operación, el lenguaje ofrece construcciones delimitadas por una finalidad específica. Su diseño requiere identificar las entidades, operaciones, datos y restricciones relevantes del dominio, así como establecer reglas que permitan reconocer e interpretar las instrucciones.

Un DSL no debe confundirse con un formulario o con una operación CRUD. Un formulario proporciona campos para capturar información y una operación CRUD permite crear, consultar, actualizar o eliminar registros. Un DSL, en cambio, define un lenguaje con reglas de formación e interpretación. Puede representar instrucciones compuestas, establecer un orden entre sus componentes y permitir que un analizador determine si una expresión pertenece al lenguaje y si cumple las condiciones correspondientes. Por tanto, su pertinencia aumenta cuando el dominio requiere expresar operaciones mediante instrucciones estructuradas y sujetas a restricciones verificables.

La aplicación de un DSL también requiere delimitar su alcance. El lenguaje debe representar únicamente las operaciones necesarias para el dominio y no pretende sustituir todos los sistemas que participan en su administración. Su función puede limitarse a describir y validar instrucciones antes de que sean procesadas por otro componente. Este enfoque permite separar la representación formal de las operaciones de los mecanismos administrativos que las originan.

En este proyecto, el dominio corresponde a los movimientos de inventario de una empresa comercial hipotética. El DSL permite representar cuatro tipos de instrucciones: entrada, salida, traslado y ajuste. Su propósito consiste en ofrecer una estructura formal para expresar esos movimientos y comprobar que cumplan las condiciones establecidas. La evaluación puede producir la aceptación estructurada de un lote o el rechazo acompañado de diagnósticos que describan los errores identificados. El lenguaje no genera código máquina ni sustituye los procedimientos administrativos de compras, ventas, almacén o contabilidad.

4. Análisis léxico y análisis sintáctico

El análisis léxico y el análisis sintáctico son etapas fundamentales del procesamiento de lenguajes. Aunque ambas participan en el reconocimiento de una entrada, sus responsabilidades son diferentes: el análisis léxico examina los caracteres y reconoce unidades elementales, mientras que el análisis sintáctico comprueba que dichas unidades se encuentren organizadas de acuerdo con una gramática.

Análisis léxico. El análisis léxico recibe una secuencia de caracteres y la examina para identificar fragmentos que correspondan a patrones reconocidos por el lenguaje. Cada fragmento identificado se denomina lexema, mientras que la categoría asignada a ese fragmento se denomina token. Aho, Lam, Sethi y Ullman (2006) explicaron que el analizador léxico agrupa los caracteres de entrada en lexemas y asigna a cada uno una categoría léxica. Por ejemplo, una secuencia de caracteres puede reconocerse como identificador, número, palabra reservada o delimitador, según los patrones definidos.

Durante este proceso, el analizador puede ignorar componentes que no intervienen en la estructura del lenguaje, como determinados espacios en blanco o comentarios, siempre que así lo establezca su especificación. También debe identificar los fragmentos que no corresponden a ningún patrón admitido y reportarlos como errores léxicos. Por tanto, esta etapa permite distinguir entre los elementos reconocidos y las secuencias de caracteres que no pertenecen a las categorías definidas.

Análisis sintáctico. El análisis sintáctico recibe la secuencia de tokens generada por el analizador léxico y verifica que su organización corresponda a las reglas de una gramática. Una gramática libre de contexto se define mediante símbolos terminales, símbolos no terminales, producciones y un símbolo inicial. Los terminales representan los elementos básicos que aparecen en las cadenas del lenguaje; los no terminales representan categorías estructurales que pueden desarrollarse mediante producciones. Las producciones establecen cómo se pueden reemplazar los no terminales por secuencias de símbolos, y el símbolo inicial identifica la categoría desde la cual comienza la generación de las estructuras válidas.

Una derivación es la secuencia de aplicaciones de producciones mediante la cual se obtiene una cadena del lenguaje. El árbol sintáctico representa gráficamente esa estructura jerárquica y permite observar cómo se relacionan los componentes de la entrada. Aho et al. (2006) describieron el análisis

sintáctico como la fase encargada de reconocer la estructura gramatical de la secuencia de tokens y construir una representación de su organización. Cuando la secuencia no satisface las producciones establecidas, se identifica un error sintáctico.

Ambas etapas mantienen una relación secuencial. El análisis léxico transforma los caracteres de entrada en una secuencia de tokens; posteriormente, el análisis sintáctico utiliza esos tokens para comprobar la estructura de la instrucción. Un error léxico se relaciona con un carácter o fragmento que no puede reconocerse según los patrones definidos, mientras que un error sintáctico aparece cuando los tokens reconocidos no cumplen el orden o la composición establecidos por la gramática. Esta separación facilita identificar la naturaleza de los errores y evita confundir problemas de reconocimiento de símbolos con problemas de estructura.

Las herramientas generadoras de analizadores permiten automatizar parte de este proceso a partir de especificaciones formales. ANTLR es una herramienta que genera analizadores léxicos y sintácticos a partir de una gramática y proporciona mecanismos para recorrer las estructuras reconocidas (Parr, 2013; Parr y Fisher, 2011). Su fundamento teórico se relaciona con los métodos de análisis predictivo y con la generación automatizada de componentes de un compilador. En este proyecto, ANTLR4 se empleó como soporte tecnológico para generar los componentes de reconocimiento del DSL; los detalles de su configuración e integración corresponden a la sección de implementación.

5. Análisis semántico y validación de reglas del dominio

El análisis semántico comprueba que una instrucción no solo esté correctamente formada, sino que también tenga un significado válido dentro del dominio al que pertenece. Por esta razón, se diferencia del análisis sintáctico: una instrucción puede cumplir todas las reglas gramaticales y, aun así, representar una operación imposible, incoherente o incompatible con los datos disponibles. Aho et al. (2006) ubicaron el análisis semántico entre las tareas de comprobación de coherencia del significado, mediante información contextual y estructuras como las tablas de símbolos. Knuth (1968), por su parte, formalizó la asociación de atributos y significados a las estructuras de las gramáticas libres de contexto.

Las reglas semánticas dependen de las características del dominio. Pueden exigir que los identificadores correspondan a elementos registrados, que las cantidades cumplan determinadas condiciones, que las ubicaciones sean válidas y que las referencias documentales sean compatibles con el tipo de operación. Para comprobar estas condiciones, el analizador necesita consultar información contextual que no siempre puede deducirse de la secuencia de tokens o de la estructura gramatical. En consecuencia, la validación semántica complementa el reconocimiento léxico y sintáctico con comprobaciones sobre el significado de la instrucción.

En la gestión de inventarios, una instrucción puede estar correctamente escrita y respetar la gramática, pero ser inválida si utiliza un producto inexistente, una ubicación no registrada, una referencia incompatible o una cantidad no permitida. De igual manera, una salida puede tener todos sus campos y delimitadores en el orden correcto, pero resultar inconsistente cuando la cantidad solicitada supera las existencias disponibles. Estas situaciones requieren información del catálogo y del estado del inventario, por lo que no pueden resolverse únicamente mediante expresiones regulares o producciones gramaticales.

La evaluación semántica también puede depender del contexto y del orden de las operaciones. Cuando un conjunto de movimientos se procesa secuencialmente, el resultado de una operación puede modificar el saldo utilizado para evaluar la siguiente. Por ello, la comprobación de una instrucción puede requerir considerar los efectos acumulados de los movimientos anteriores dentro del lote. Esta evaluación debe distinguirse de una modificación definitiva del inventario: comprobar el estado proyectado de las existencias no implica necesariamente actualizar el registro original.

En el proyecto, el análisis semántico se utiliza para evaluar la coherencia de los movimientos de entrada, salida, traslado y ajuste con los datos de referencia y las condiciones del inventario. Su función consiste en identificar inconsistencias que no pueden detectarse únicamente mediante el reconocimiento de tokens o la comprobación gramatical. La definición específica de las reglas aplicadas corresponde al desarrollo del proyecto y no se detalla en este apartado teórico.

Los cinco conceptos desarrollados proporcionan el fundamento para relacionar la gestión de inventarios con los principios de la teoría de compiladores. La gestión de inventarios permite comprender el dominio y las consecuencias de la inexactitud de los registros; los lenguajes formales y los autómatas sustentan la definición y el reconocimiento de patrones; los lenguajes específicos de dominio permiten representar operaciones mediante instrucciones especializadas; y los análisis léxico y sintáctico comprueban sus componentes y estructura. Finalmente, el análisis semántico permite evaluar si las instrucciones reconocidas cumplen las restricciones propias del inventario. En conjunto, estos fundamentos justifican la separación entre el reconocimiento formal de las instrucciones y la comprobación contextual de los movimientos, sin anticipar los resultados de la implementación.

La Tabla 6 sintetiza los fundamentos teóricos desarrollados y su relación con el proyecto. La correspondencia presentada permite distinguir la función conceptual de cada tema y el aspecto de la solución que se sustenta en él.

Tabla 6.

Relación entre los fundamentos teóricos y su aplicación en el proyecto

Fundamento teórico	**Concepto principal**	**Función general**	**Aplicación posterior en el proyecto**
Gestión de inventarios e IRI	Correspondencia entre existencias físicas y registros	Comprender la confiabilidad de la información y las consecuencias de las discrepancias	Delimitación de los movimientos y de las inconsistencias que deben identificarse.
Lenguajes formales y autómatas	Reconocimiento de cadenas pertenecientes a un lenguaje	Definir formalmente patrones y mecanismos de reconocimiento	Especificación de los elementos léxicos y su reconocimiento.

DSL	Lenguaje especializado para un dominio	Representar operaciones mediante instrucciones propias del ámbito	Representación formal de entradas, salidas, traslados y ajustes.
Análisis léxico y sintáctico	Reconocimiento de tokens y comprobación gramatical	Identificar componentes y verificar la estructura de las instrucciones	Generación del lexer y parser, y detección de errores léxicos y sintácticos.
Análisis semántico	Comprobación contextual de reglas del dominio	Determinar si una instrucción estructurada tiene significado válido	Validación de datos de referencia, cantidades, ubicaciones y consistencia de movimientos.

Nota. Elaboración propia a partir de los fundamentos teóricos citados en el apartado.

METODOLOGÍA DE IMPLEMENTACIÓN

La selección de un modelo de desarrollo permitió organizar las actividades del proyecto y establecer una secuencia de trabajo coherente con sus objetivos y productos. Debido a que la propuesta comprendió la formalización de un lenguaje, el diseño de sus mecanismos de análisis, la implementación de la validación y la evaluación de su funcionamiento, resultó necesario adoptar un enfoque que facilitara la planificación de las fases y la trazabilidad de sus entregables. Para determinar el modelo más adecuado, se consideraron tres alternativas: el modelo en cascada o Waterfall, Scrum y el modelo de prototipado. La comparación tomó en cuenta sus características, ventajas, limitaciones y correspondencia con el carácter académico y secuencial del proyecto.

Scrum

Scrum es un marco de trabajo ágil que organiza el desarrollo de productos mediante ciclos iterativos e incrementales. Schwaber y Sutherland (2020) lo describen como un marco en el que un equipo trabaja sobre un objetivo de producto, organiza el trabajo pendiente en un Product Backlog y desarrolla incrementos mediante periodos breves denominados Sprints. El marco establece responsabilidades, eventos y artefactos para facilitar la transparencia, la inspección y la adaptación del trabajo.

Una de sus principales ventajas es la posibilidad de recibir retroalimentación frecuente y ajustar las prioridades durante el desarrollo. La entrega de incrementos permite revisar el producto de manera continua y adaptar el trabajo ante cambios en las necesidades. Sin embargo, su aplicación requiere una organización de equipo que asuma las responsabilidades y actividades del marco, así como una disponibilidad suficiente para revisar los incrementos y priorizar el trabajo pendiente. En un proyecto pequeño o individual, la aplicación formal de sus roles y eventos puede representar una carga organizativa que no necesariamente aporta beneficios proporcionales.

Para este proyecto, Scrum habría permitido organizar entregas parciales del analizador y de la interfaz. Sin embargo, no fue el modelo principal seleccionado, porque la estructura académica requería desarrollar y documentar fases diferenciadas, y las actividades presentaban dependencias formales entre sí. La necesidad de establecer primero los patrones léxicos, después la gramática y posteriormente la interpretación semántica favoreció una organización secuencial. Por ello, se consideró que Scrum no representaba la alternativa principal más adecuada para estructurar la documentación y los entregables del proyecto.

Modelo en cascada (Waterfall)

El modelo en cascada, conocido también como Waterfall, organiza el desarrollo de software mediante fases sucesivas en las que los productos obtenidos en una etapa sirven como base para la siguiente. Royce (1970) describió un modelo de desarrollo que contempla actividades relacionadas con la definición de requisitos, el diseño, la implementación, la verificación y el mantenimiento. Pressman y Maxim (2020) presentan el modelo secuencial como un enfoque en el que las actividades se planifican y documentan por etapas, procurando completar los productos de cada fase antes de avanzar.

Entre sus ventajas se encuentran la organización explícita de las actividades, la documentación de los entregables, la trazabilidad entre requisitos y productos, y la posibilidad de evaluar el avance mediante resultados definidos para cada etapa. Estas características resultan útiles en proyectos académicos que deben presentar evidencias diferenciadas de análisis, diseño, desarrollo y pruebas. Sin embargo, el modelo también presenta limitaciones: los cambios en los requisitos pueden generar revisiones

costosas y algunos errores de definición pueden detectarse cuando las fases posteriores ya dependen de ellos. Por tanto, su aplicación requiere revisar los productos intermedios y permitir correcciones controladas cuando se identifiquen inconsistencias.

En este proyecto, Waterfall resultó pertinente porque el trabajo se organizó alrededor de productos con dependencias técnicas: la identificación del problema y de los requisitos precedió al modelamiento formal; la definición de los componentes del lenguaje permitió establecer su gramática; y la especificación gramatical sirvió como base para generar los analizadores y desarrollar la validación semántica. La secuencia también permitió relacionar cada etapa con documentos y evidencias verificables. No obstante, el proceso no se consideró estrictamente irreversible, debido a que la revisión de la gramática y de la documentación requirió ajustes durante el desarrollo.

Modelo de prototipado

El modelo de prototipado consiste en construir una versión preliminar de un sistema con el propósito de explorar, aclarar o validar requisitos antes de consolidar la solución. Pressman y Maxim (2020) explican que el prototipo puede evaluarse y refinarse de acuerdo con la información obtenida durante su revisión; posteriormente, puede evolucionar hacia el producto final o descartarse cuando haya cumplido su función de aclaración.

Entre sus ventajas se encuentran la posibilidad de visualizar tempranamente una solución, detectar dificultades de interacción y reducir la incertidumbre cuando los requisitos no están suficientemente definidos. También facilita la revisión de interfaces y de flujos de uso antes de realizar una implementación más completa. No obstante, puede generar ampliaciones no previstas del alcance si cada revisión incorpora nuevas funcionalidades. Asimismo, cuando se concentra excesivamente en una representación preliminar de la interfaz, puede dejar en segundo plano la documentación formal y la especificación técnica del sistema.

En el proyecto, el prototipado habría sido pertinente para explorar la presentación web de las instrucciones y de los diagnósticos. Sin embargo, no resultó suficiente como modelo principal, porque

el propósito central no consistió únicamente en construir una interfaz, sino en formalizar un lenguaje y desarrollar mecanismos de reconocimiento y validación. La corrección de la estructura del lenguaje, la definición de sus reglas y la comprobación de su comportamiento requerían entregables formales que no podían sustituirse por la evaluación visual de un prototipo. Por ello, el prototipado se consideró una alternativa útil para aspectos puntuales de interacción, pero no como el enfoque general de implementación.

Justificación de la metodología seleccionada

La Tabla 7 compara Waterfall, Scrum y el modelo de prototipado a partir de criterios relacionados con la organización del trabajo, la gestión de requisitos, la documentación y la adecuación al alcance académico. La comparación permite justificar la elección del modelo principal sin asumir que una alternativa sea superior en todos los contextos.

Tabla 7

Comparación de modelos de desarrollo considerados

Criterio	Waterfall	Scrum	Prototipado
Organización de actividades	Fases secuenciales con entregables definidos.	Trabajo organizado en iteraciones e incrementos.	Construcción, evaluación y refinamiento de versiones preliminares.
Planificación	Planificación anticipada de fases y productos.	Planificación adaptada a cada Sprint y al backlog.	Planificación orientada a explorar y validar aspectos de la solución.
Estabilidad de requisitos	Se adapta mejor cuando los requisitos pueden definirse anticipadamente.	Admite cambios y repriorización durante el desarrollo.	Es útil cuando los requisitos iniciales necesitan aclararse.

Documentación	Favorece la documentación de cada fase y sus entregables.	La documentación se ajusta a las necesidades del producto y del equipo.	Puede concentrarse en los prototipos y en la retroalimentación obtenida.
Retroalimentación	Se incorpora mediante revisiones y verificaciones por etapa.	Es frecuente y forma parte de la inspección y adaptación.	Se obtiene mediante la evaluación de las versiones preliminares.
Cambios durante el desarrollo	Pueden requerir revisar documentos y fases anteriores.	Se gestionan mediante la adaptación del backlog y los siguientes Sprints.	Se incorporan durante el refinamiento del prototipo.
Adecuación al proyecto académico	Alta, porque permite relacionar análisis, diseño, desarrollo y pruebas con entregables verificables.	Posible, aunque exige organizar el trabajo académico mediante iteraciones y responsabilidades del marco.	Parcial, porque resulta útil para explorar la interfaz, pero no cubre por sí solo la formalización del lenguaje.
Relación con el modelamiento formal	Facilita establecer una secuencia entre especificación, gramática, análisis e implementación.	Permite desarrollar el modelamiento de forma incremental, con revisiones frecuentes.	Permite explorar representaciones, pero no sustituye la especificación formal del lenguaje.
Ventajas para el proyecto	Trazabilidad documental, fases identificables y correspondencia con los entregables de la guía.	Flexibilidad para incorporar retroalimentación y organizar incrementos funcionales.	Evaluación temprana de la interacción y de la presentación de resultados.
Limitaciones para el proyecto	Puede dificultar cambios amplios cuando ya existen productos dependientes de fases anteriores.	Su aplicación formal puede añadir actividades organizativas innecesarias para el alcance y tamaño del proyecto.	No garantiza por sí solo la definición rigurosa de la gramática ni de las reglas semánticas.

Nota. Elaboración propia a partir de Royce (1970), Pressman y Maxim (2020) y Schwaber y Sutherland (2020).

El modelo en cascada o Waterfall se seleccionó como referencia principal para organizar la implementación del proyecto debido a la dependencia lógica entre sus actividades y a la necesidad de documentar productos verificables en cada fase. El trabajo comenzó con la identificación del problema relacionado con la representación y validación de movimientos de inventario, así como con la delimitación de los requisitos y del alcance del caso hipotético. Posteriormente, se desarrolló el modelamiento formal del lenguaje, que comprendió la definición de sus componentes y de las reglas necesarias para reconocer las instrucciones. Sobre esta base se diseñaron los mecanismos de análisis léxico, sintáctico y semántico, junto con la arquitectura de la solución.

La fase de desarrollo se apoyó en ANTLR4 para generar los componentes de reconocimiento y en Python para integrar el procesamiento de las instrucciones y las reglas semánticas. Flask permitió proporcionar una interfaz web para ingresar instrucciones y presentar los resultados. Después de la implementación se realizaron pruebas manuales de regresión y se recopilaron evidencias para comprobar los comportamientos de aceptación y rechazo, así como la conservación del archivo original de existencias. Finalmente, se publicó una demostración de la aplicación mediante GitHub y Render, con el propósito de facilitar el acceso al prototipo, sin que ello representara su incorporación a la operación de una empresa real.

La elección de Waterfall no implica que las fases se hayan ejecutado de manera absolutamente irreversible. Durante el trabajo se realizaron revisiones documentales y técnicas cuando se identificaron inconsistencias entre la especificación del lenguaje, la gramática y los componentes implementados. Estos ajustes permitieron corregir productos previos antes de continuar con las actividades dependientes. En consecuencia, se aplicó una organización predominantemente secuencial, acompañada de revisiones controladas entre fases, y no una ejecución rígida en la que se prohibiera regresar a una etapa anterior.

El despliegue se consideró una actividad de demostración posterior a las pruebas, no una puesta en operación institucional. El proyecto no incorporó el prototipo a los procesos reales de una empresa

comercial ni implementó un servicio de mantenimiento institucional. Por tanto, las actividades de mantenimiento y soporte se consideran trabajo futuro, orientado a posibles correcciones, mejoras y ampliaciones de la solución. Esta delimitación mantiene la correspondencia entre el modelo de desarrollo seleccionado y el alcance efectivamente alcanzado.

La Tabla 8 presenta la correspondencia entre las fases del modelo Waterfall y las actividades desarrolladas en el proyecto. Se incluyen las etapas de análisis, diseño, desarrollo y pruebas, además del despliegue de demostración y del mantenimiento considerado como trabajo futuro. Esta clasificación permite diferenciar los productos efectivamente elaborados de las actividades que todavía no forman parte del alcance ejecutado.

Tabla 8

Aplicación del modelo Waterfall al proyecto

Fase	Propósito	Actividades realizadas	Producto o evidencia	Estado
Análisis	Delimitar el problema, el dominio y los requisitos de la solución.	Identificación del problema de validación de movimientos; descripción del proceso de inventarios; delimitación del caso hipotético; identificación de requisitos, tipos de errores y alcance funcional; modelamiento inicial del proceso.	Descripción del problema, requisitos, clasificación de errores, definición del alcance y documentación del proceso.	Realizada.

Diseño	Especificar formalmente el lenguaje y la estructura de la solución.	Definición del DSL; identificación de tokens y lexemas; elaboración de expresiones regulares; construcción de AFND y AFD; formulación de la GLC; elaboración de derivaciones y árbol sintáctico; diseño de la arquitectura y de los flujos de validación.	Especificación del lenguaje, expresiones regulares, autómatas, gramática, derivaciones, árbol sintáctico y diagramas de diseño.	Realizada, con revisiones documentales y técnicas.
Desarrollo	Implementar los componentes de reconocimiento, interpretación y validación.	Elaboración de Inventario.g4; generación del lexer y parser mediante ANTLR 4.13.2; integración de VisitorMovimientos; implementación de las reglas semánticas M01–M09; procesamiento en Python; integración de la interfaz Flask; lectura de datos JSON y ejecución desde consola.	Gramática, componentes generados, código Python, visitor, validador semántico, interfaz Flask y salidas JSON.	Realizada.
Pruebas	Comprobar el comportamiento del analizador y la consistencia de sus resultados.	Ejecución manual de casos de regresión; comprobación de entradas aceptadas y rechazadas; revisión de errores léxicos, sintácticos y semánticos; aplicación de checklist; recopilación de evidencias; verificación de que el archivo	Matriz de casos de regresión, registros de ejecución, checklist, capturas y comprobación de integridad del archivo de existencias.	Realizada mediante pruebas manuales. La automatización adicional de pruebas quedó pendiente.

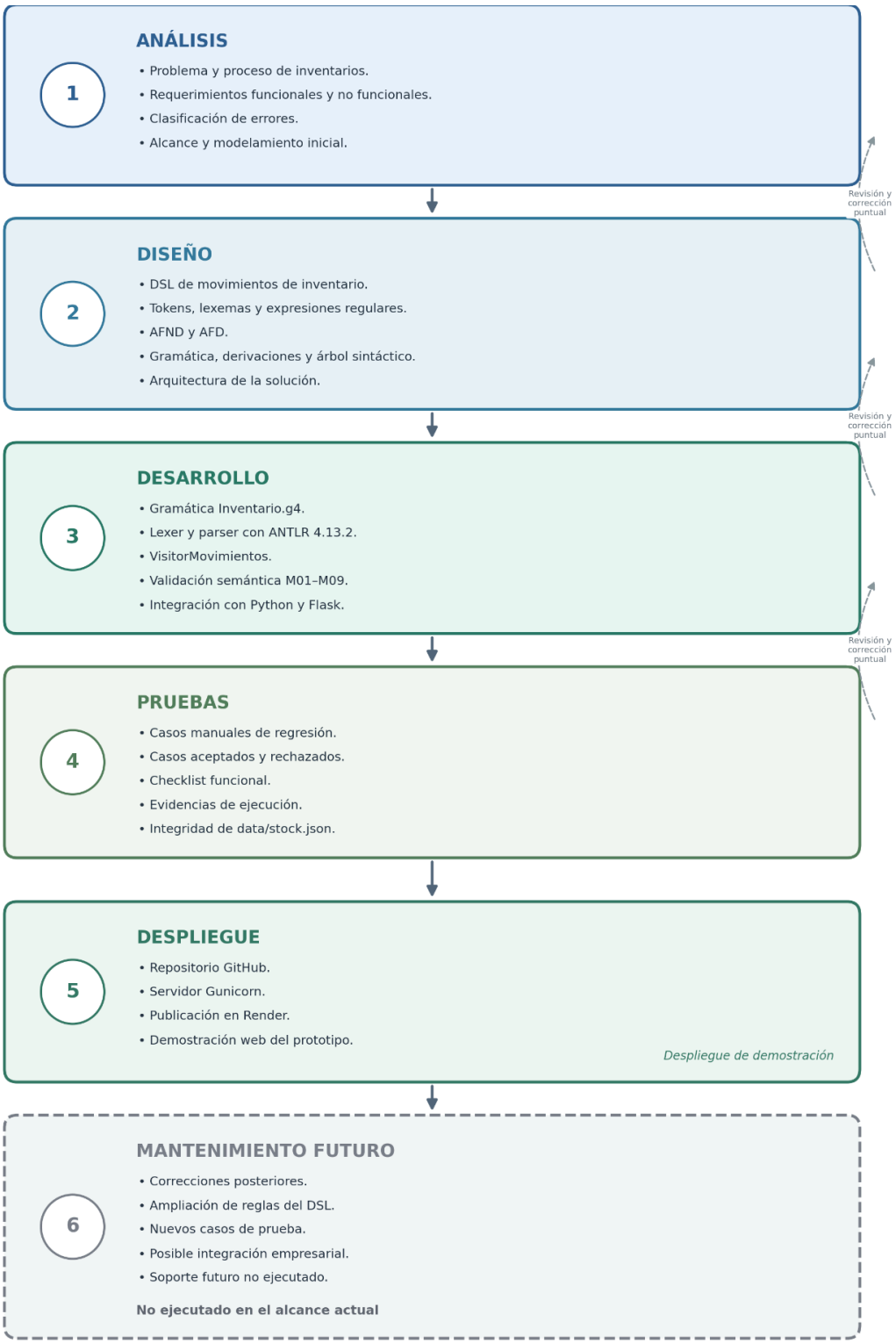
		data/stock.json no se modifica.		
Despliegue	Publicar una versión accesible para demostración.	Publicación del repositorio en GitHub; configuración del servicio de ejecución mediante Gunicorn; despliegue de la aplicación en Render; comprobación de acceso a la demostración web.	Repositorio publicado y aplicación de demostración disponible en https://inventario-dsl.onrender.com/.	Realizado como demostración.
Mantenimiento y soporte	Mantener y mejorar la solución después de su entrega.	Se consideran futuras correcciones, mejoras de validación, ampliaciones del lenguaje y actividades de soporte posteriores. No se ejecutó mantenimiento institucional ni soporte a usuarios empresariales.	Propuesta de trabajo futuro; no existe evidencia de operación institucional o mantenimiento posterior.	Propuesto como trabajo futuro.

Nota. Elaboración propia a partir de las actividades y evidencias del proyecto. El estado «realizada» se refiere al trabajo académico y al prototipo; no implica implementación en una empresa real.

La Figura 4 representa la organización secuencial de las fases del modelo Waterfall aplicadas al proyecto. Se muestran los principales productos de cada etapa y se diferencia el despliegue de demostración del mantenimiento, que se considera una actividad futura.

Figura 4

Aplicación secuencial del modelo Waterfall en el proyecto



Nota. Elaboración propia a partir de las fases y actividades realizadas en el proyecto.

Como se presenta en la Figura 4, el proyecto siguió una organización predominantemente secuencial, en la que cada fase produjo elementos necesarios para continuar con la siguiente. Sin embargo,

durante el proceso se realizaron revisiones puntuales para corregir inconsistencias en la especificación, la gramática y la documentación. El despliegue se efectuó con fines demostrativos y no representó la incorporación del prototipo a una organización empresarial. Asimismo, el mantenimiento y el soporte se mantienen como actividades futuras, debido a que no se desarrolló una etapa de operación institucional.

La metodología seleccionada permitió organizar el trabajo mediante fases relacionadas con la identificación del problema, la formalización del lenguaje, el diseño de los mecanismos de análisis, la implementación y la evaluación funcional. La aplicación de Waterfall proporcionó una estructura para documentar los productos de cada etapa y mantener la correspondencia con los objetivos establecidos, mientras que las revisiones puntuales permitieron corregir inconsistencias detectadas durante el proceso. A partir de esta organización, el desarrollo del proyecto se documentará mediante cuatro fases principales: Análisis, Diseño, Desarrollo y Pruebas. El despliegue de la aplicación se considerará dentro de la descripción de la implementación técnica, como una demostración de la solución y no como una operación empresarial.

DESARROLLO DEL PROYECTO

El desarrollo del proyecto se organizó tomando como referencia el modelo de ciclo de vida en cascada (Waterfall), de manera que las actividades se documentaron en fases secuenciales y sus productos sirvieron como insumos para las etapas posteriores. Esta sección comprende las fases de Análisis, Diseño, Desarrollo y Pruebas, conforme a la estructura establecida para el trabajo. El caso empresarial se planteó como un escenario hipotético de una empresa comercial de Lima y se trabajó con datos simulados, por lo que la solución no representó una instalación ni una intervención en una organización real. En ese contexto, se implementó un prototipo funcional orientado a recibir lotes de movimientos de inventario, reconocer su estructura y evaluar su coherencia mediante validaciones léxicas, sintácticas y semánticas. La documentación de cada fase permite distinguir el modelamiento formal del lenguaje, la automatización de sus analizadores y la integración de los componentes computacionales, sin atribuir al prototipo funciones ajenas a su alcance.

1. ANÁLISIS

La fase de Análisis tuvo como propósito delimitar el problema computacional que debía abordar el prototipo, identificar las necesidades de validación vinculadas con el registro de movimientos de inventario y establecer los requisitos que orientarían su construcción. El problema se centró en determinar si una instrucción escrita mediante un lenguaje específico del dominio (DSL) representaba correctamente una operación de inventario y si sus datos eran coherentes con los catálogos y las existencias simuladas. Para ello, se definió el procesamiento de lotes que podían contener uno o más movimientos de tipo ENTRADA, SALIDA, TRASLADO o AJUSTE, y se distinguieron tres niveles de análisis: léxico, sintáctico y semántico. El alcance comprendió la recepción de instrucciones mediante una interfaz web o la consola, la comprobación de su estructura, la evaluación de reglas del dominio y la presentación de los resultados. Asimismo, se identificaron los requisitos funcionales y no funcionales y se estableció el modelamiento formal inicial del lenguaje, relacionado con el proceso de inventarios del caso hipotético. Esta delimitación permitió especificar qué debía reconocer el analizador y qué condiciones debían cumplirse antes de considerar aceptado un lote.

1.1. Requerimientos funcionales

Los requerimientos funcionales se revisaron considerando las funciones que efectivamente ofrece el prototipo y la separación entre sus dos modalidades de ejecución: la interfaz web y la consola. La implementación utiliza la gramática grammar/Inventario.g4, los componentes generados por ANTLR 4.13.2, VisitorMovimientos y ValidadorSemantico. En la interfaz web se puede escribir una instrucción o cargar un archivo .dsl o .txt; el procesamiento continúa por las fases léxica, sintáctica y semántica, y presenta el resultado en la página. La ejecución por consola conserva la capacidad de procesar archivos y generar una salida JSON local cuando el lote es aceptado.

Para asignar la prioridad se utilizó un criterio de importancia funcional: se considera alta cuando la función es necesaria para recibir, analizar, validar o resolver la aceptación integral de un lote; se considera media cuando complementa el acceso o la ejecución del prototipo, pero no constituye una

regla central de validación. Esta clasificación es una decisión de análisis del proyecto y no una propiedad técnica definida por ANTLR o Flask.

Tabla 9

Requerimientos funcionales del analizador DSL

Código	Requerimiento funcional	Descripción	Prioridad	Componente responsable	Evidencia verificable
RF01	Ingresar instrucciones DSL desde la interfaz web	Permite escribir o pegar el contenido de un lote en el formulario de la aplicación web para iniciar su análisis.	Alta	app/routes.py, app/services/analizador_service.py e interfaz web	Captura 08_interfaz_principal.png y capturas de ejecución web.
RF02	Cargar archivos .dsl o .txt	Permite seleccionar y leer archivos con las extensiones admitidas para procesar su contenido como entrada del analizador.	Alta	app/routes.py y app/services/analizador_service.py	Captura de la interfaz de carga y evidencia de procesamiento de archivos.
RF03	Reconocer tokens y detectar errores léxicos	Ejecuta el lexer generado a partir de Inventario.g4, reconoce los componentes léxicos y detecta los errores L01-L05 antes	Alta	generated/InventarioLexer.py y servicio de análisis	Captura 10_tokens_lexemas.png y captura 14_error_lexico_l01.png.

		de continuar con el parser.			
RF04	Comprobar la estructura sintáctica	Ejecuta el parser y verifica que la secuencia de tokens corresponda con las producciones de la gramática, clasificando los errores previstos S01-S08.	Alta	generated/InventarioParser.py y listener sintáctico	Captura 15_error_sintactico_s01.png y casos de casos/sintacticos/.
RF05	Construir y mostrar el árbol sintáctico	Obtiene el árbol de análisis generado por ANTLR y lo presenta como parte de la información del lote procesado.	Alta	generated/InventarioParser.py y app/services/analizador_service.py	Captura 11_arbol_sintactico.png.
RF06	Extraer movimientos mediante VisitorMovimientos	Recorre el árbol sintáctico válido y transforma los bloques reconocidos en objetos MovimientoInventario con sus campos y posiciones.	Alta	src/visitor_movimientos.py	Captura 12_movimientos_extraidos.png y código de VisitorMovimientos.

RF07	Aplicar reglas semánticas M01-M09	Comprueba la existencia de productos y ubicaciones, la cantidad, el stock, las ubicaciones de traslado, las referencias, el tipo de ajuste, los motivos y los límites documentados.	Alta	src/validador_semantico.py y archivos JSON de data/	Captura 16_error_semantico_m01.png y archivos de casos/semanticos/.
RF08	Presentar diagnósticos con fase, código, ubicación, lexema y descripción	Informa la fase y el código del error, la línea y columna, el lexema asociado y una descripción del problema detectado.	Alta	Servicio de análisis, listener sintáctico, lexer y validador semántico	Capturas 14_error_lexico_l01.png, 15_error_sintactico_s01.png y 16_error_semantico_m01.png.
RF09	Presentar o generar JSON para lotes aceptados	En la web presenta el JSON de aceptación en la respuesta de la página; desde la consola genera un archivo JSON local en el directorio de salida.	Alta	app/services/analizador_service.py y src/main.py	Captura 13_json_generado.png y evidencia de respuesta JSON en la interfaz web.
RF10	Rechazar integralmente lotes inconsistentes	Impide aceptar parcialmente un lote cuando alguno de sus movimientos incumple las	Alta	src/validador_semantico.py y servicio de análisis	Casos de rechazo semántico, incluido rechazo_integral.dsl, y captura de un lote rechazado.

		validaciones establecidas.			
RF11	Evaluar movimientos sobre una copia temporal sin modificar data/stock.json	Calcula los efectos de los movimientos sobre una copia temporal del stock inicial y conserva intacto el archivo JSON original.	Alta	src/validador_semantico.py	Captura 17_stock_sin_modificar.png y código que utiliza copy.deepcopy().
RF12	Ejecutar el análisis desde consola	Permite procesar un archivo DSL mediante el módulo principal de Python y obtener el estado de aceptación o rechazo y, si corresponde, una salida JSON local.	Media	src/main.py	Captura 06_ejecucion_consola_valida.png y comandos documentados en README.md.

Nota. Elaboración propia a partir de la revisión de la gramática, los componentes Python, la documentación del repositorio y las evidencias de ejecución. Las prioridades representan la clasificación funcional adoptada para el proyecto.

1.2. Requerimientos no funcionales

Los requerimientos no funcionales se formularon a partir de atributos observables en el prototipo. Su evaluación no supone que todos se encuentren plenamente garantizados: algunos cuentan con evidencia directa de implementación, mientras que otros solo pueden considerarse parcialmente atendidos porque no se realizaron mediciones específicas ni evaluaciones formales de calidad.

Tabla 10

Requerimientos no funcionales del analizador DSL

Código	Categoría	Requerimiento	Criterio de verificación	Evidencia	Estado
RNF 01	Claridad de diagnósticos	Los errores detectados deben presentar código, fase, línea, columna, lexema y descripción comprensible.	Revisar que los diagnósticos de las tres fases incluyan los datos de identificación previstos.	Capturas de errores L01, S01 y M01; salida de los diagnósticos.	Cumplido
RNF 02	Integridad del stock	La evaluación de un lote no debe modificar el archivo data/stock.json.	Comparar el contenido del archivo antes y después de procesar lotes aceptados y rechazados.	Captura 17_stock_sin_modificar.png; uso de una copia temporal mediante copy.deepcopy().	Cumplido
RNF 03	Reproducibilidad	Un mismo archivo DSL, procesado con los mismos datos maestros y stock inicial, debe producir el mismo estado esperado.	Repetir los casos fijos y comparar sus códigos de salida con los valores esperados.	Registro de 31 casos manuales de regresión: 6 válidos, 5 léxicos, 8 sintácticos y 12 semánticos.	Cumplido en los casos verificados

RNF 04	Modularidad	La gramática, el análisis generado, la extracción de movimientos y la validación semántica deben encontrarse separados en componentes identificables.	Inspeccionar la estructura del repositorio y las responsabilidades de los módulos.	grammar/Inventario.g4, generated/, src/visitor_movimientos.py y src/validador_semantico.py.	Cumplido
RNF 05	Mantenibilidad	La separación de responsabilidades y el control de versiones deben facilitar la localización y modificación de los componentes del prototipo.	Revisar la organización de los módulos y el historial de cambios del repositorio.	Estructura del proyecto y repositorio GitHub. No se aplicó una métrica formal de mantenibilidad.	Parcial
RNF 06	Portabilidad	El prototipo debe poder ejecutarse en un entorno que disponga de una versión compatible de Python y del runtime de ANTLR requerido.	Verificar la instalación de dependencias y la ejecución del proyecto en los entornos documentados.	requirements.txt, documentación de instalación y despliegue.	Parcial

RNF 07	Usabilidad	La interfaz web debe permitir introducir o cargar un lote y consultar el resultado del análisis sin ejecutar manualmente comandos de consola.	Comprobar el ingreso de texto, la carga de archivos y la visualización de resultados en la interfaz.	Capturas de la interfaz principal, lote aceptado y lote rechazado.	Cumplido funcionalmente; sin evaluación formal de usuarios
RNF 08	Trazabilidad mediante Git	Los cambios del código y de los archivos del proyecto deben quedar asociados al repositorio y a su historial de versiones.	Inspeccionar los commits y los archivos publicados en el repositorio.	Repositorio GitHub y captura 18_repositorio_actualizado.png.	Cumplido
RNF 09	Accesibilidad de demostración	La aplicación debe contar con una dirección web desde la cual pueda consultarse la demostración publicada.	Comprobar el acceso a la dirección pública y la carga de la interfaz.	Capturas de Render y de la aplicación publicada.	Cumplido en la verificación documentada

Nota. Elaboración propia. Los estados se asignaron según la evidencia disponible y no equivalen a una certificación de calidad del software.

La reproducibilidad se considera cumplida únicamente para los casos fijos que fueron ejecutados y contrastados. La batería de 31 escenarios corresponde a pruebas manuales realizadas desde PowerShell y no a una suite automatizada de pytest. De igual manera, la modularidad se sustenta en la separación observable de los componentes, pero no implica que se haya realizado una auditoría formal de calidad del código.

En cuanto a la portabilidad, la documentación registra un entorno local con Windows 11, Python 3.14.3, Eclipse Temurin OpenJDK 25.0.1 y ANTLR 4.13.2. También especifica los requisitos para regenerar los módulos del analizador y diferencia esa operación de la ejecución ordinaria, para la cual los archivos generados ya se encuentran incluidos en el repositorio. Por ello, la portabilidad se considera parcial y limitada a los entornos documentados, sin afirmar compatibilidad universal.

La publicación en Render permite acceder a una demostración web del prototipo, pero no constituye una garantía de disponibilidad permanente, continuidad operativa ni nivel de servicio. La accesibilidad indicada en la tabla se refiere únicamente a la existencia de una dirección pública y a la verificación documentada de su funcionamiento en el momento de las evidencias.

1.3. Recursos tecnológicos utilizados

El desarrollo se realizó en un equipo personal con Windows 11, utilizando Python 3.14.3 como entorno de ejecución local. Para la generación del lexer, parser y visitor se empleó ANTLR 4.13.2, cuyo funcionamiento requiere Java y el archivo JAR de la misma versión cuando se necesita regenerar los módulos a partir de la gramática. La ejecución ordinaria utiliza los componentes generados que se encuentran en el repositorio y el runtime de ANTLR para Python, por lo que no requiere regenerar la gramática en cada ejecución.

La interfaz web se implementó con Flask y se integró con Gunicorn para su publicación en Render. Git se utilizó para el control de versiones y GitHub para alojar el repositorio del proyecto. La interacción con la aplicación publicada se realizó mediante un navegador web. Para las actividades de ejecución y verificación local se documentó el uso de PowerShell. No se incorpora una versión específica del navegador ni un editor de código en esta descripción, porque no se dispone de evidencia suficiente para fijarlos como recursos comprobados.

1.4. Modelamiento formal del proceso

El modelamiento formal tuvo como propósito establecer una representación precisa de los movimientos de inventario que pudiera ser reconocida y procesada por un analizador. Para ello, se definió un lenguaje específico del dominio cuyo documento contiene un bloque principal LOTE, dentro del cual se declara uno o más movimientos. Cada movimiento representa una operación de tipo ENTRADA, SALIDA, TRASLADO o AJUSTE, y posee campos obligatorios organizados en un orden determinado, además de un campo opcional OBSERVACION.

La representación formal distingue entre la forma y el significado de las instrucciones. La forma corresponde a los caracteres, las secuencias de símbolos, los patrones léxicos y la organización gramatical que debe cumplir el documento. El significado corresponde a las condiciones del proceso de inventarios que deben verificarse una vez reconocida la estructura. Por ello, el análisis léxico identifica las unidades del lenguaje; el análisis sintáctico comprueba su organización conforme a la gramática; y el análisis semántico verifica la coherencia de los valores con los datos maestros y las existencias simuladas.

El lote constituye la unidad de procesamiento. Los movimientos se evalúan en el orden en que aparecen y sus efectos se proyectan sobre una copia temporal del stock inicial. Si un movimiento incumple una regla, no se aplica a esa copia y el lote completo se rechaza. Si no se detectan errores, se obtiene una representación JSON con los movimientos interpretados y el stock proyectado. Este procedimiento no modifica el archivo original data/stock.json ni confirma una transacción en un inventario empresarial real.

Tabla 11

Componentes del modelamiento formal del DSL

Componente	Definición	Aplicación en el proyecto	Evidencia posterior

Alfabeto	Conjunto finito de símbolos que pueden integrar las cadenas del lenguaje.	Reúne los caracteres estructurales, numéricos, identificadores, espacios y caracteres permitidos en los valores textuales.	Apartado 1.5 y gramática Inventario.g4.
Cadenas	Secuencias finitas de símbolos pertenecientes al alfabeto.	Representan los documentos DSL que contienen un bloque LOTE y sus movimientos.	Apartado 1.6 y archivos .dsl de ejemplo.
Lenguaje	Subconjunto de las cadenas sobre un alfabeto que satisface las reglas definidas.	Comprende los documentos que cumplen las condiciones léxicas y sintácticas del DSL.	Apartado 1.6 y regla inicial documento.
Expresiones regulares	Notaciones que describen patrones de cadenas reconocibles por un autómata finito.	Especifican los patrones de enteros, identificadores y cadenas de texto, entre otros elementos léxicos.	Apartado 1.7 y reglas del lexer.
AFND	Autómata finito no determinista que reconoce un lenguaje regular.	Representa formalmente patrones léxicos seleccionados del DSL.	Apartado 1.8 y figuras de autómatas.
AFD	Autómata finito determinista equivalente a un AFND para el mismo lenguaje.	Representa los patrones seleccionados mediante transiciones deterministas.	Apartado 1.9 y figuras de autómatas.
Tokens	Categorías léxicas asignadas a las secuencias reconocidas.	Identifican palabras reservadas, campos, cadenas, enteros y delimitadores.	Apartado posterior de Diseño del análisis léxico.

Lexemas	Secuencias concretas de caracteres que corresponden a tokens.	Son los valores efectivamente escritos en el documento, como ENTRADA, CANTIDAD o 10.	Apartado posterior de Diseño del análisis léxico.
GLC	Gramática formal cuyas producciones describen la estructura de las cadenas del lenguaje.	Define la composición del lote, los movimientos y sus campos en un orden específico.	Apartado posterior de Diseño del análisis sintáctico y Inventario.g4.
Derivación	Secuencia de aplicaciones de producciones que permite obtener una cadena desde el símbolo inicial.	Explica cómo se genera formalmente la estructura de un lote válido.	Apartado posterior de Diseño del análisis sintáctico.
Árbol sintáctico	Representación jerárquica de las aplicaciones de las producciones gramaticales.	Corresponde al árbol de análisis (parse tree) producido por ANTLR.	Apartado posterior de Diseño y evidencia 11_arbol_sintactico.png.
Reglas semánticas	Condiciones que determinan la coherencia de los valores y operaciones dentro del dominio.	Comprueban productos, ubicaciones, cantidades, stock, traslados, referencias y ajustes mediante M01-M09.	Implementación de src/validador_semantico.py y evidencia de validación.

Nota. Elaboración propia. La tabla presenta la función de cada componente dentro del modelamiento; su especificación detallada se desarrolla en los apartados correspondientes.

1.4.1. Criterios de aceptación y rechazo del lote

El prototipo considera el lote como la unidad de procesamiento. Su aceptación no depende únicamente de que cada instrucción tenga una estructura reconocible, sino de que todas las operaciones superen consecutivamente el análisis léxico, el análisis sintáctico y la validación

semántica. Por esta razón, el procesamiento se realiza en etapas y en el orden en que aparecen los movimientos dentro del lote.

En primer lugar, el análisis léxico identifica los componentes de la entrada y detecta los errores L01-L05. Si se encuentra alguno de estos errores, el lote se rechaza y no se continúa con el análisis sintáctico. Cuando no existen errores léxicos, el parser comprueba que la secuencia de tokens cumpla la estructura definida en Inventario.g4. Los incumplimientos estructurales se clasifican mediante los diagnósticos S01-S08.

Si la estructura es válida, VisitorMovimientos recorre el árbol sintáctico y extrae los movimientos para representarlos mediante estructuras de Python. Posteriormente, ValidadorSemantico comprueba las reglas M01-M09 utilizando los datos JSON de productos, ubicaciones, referencias, motivos y stock inicial. Esta etapa verifica la existencia y compatibilidad de los datos, las cantidades, las restricciones de los traslados, los ajustes y la disponibilidad acumulada.

La evaluación semántica utiliza una copia temporal del stock inicial. Los movimientos se procesan secuencialmente: cada operación válida actualiza la copia temporal y esta nueva situación se utiliza para evaluar las operaciones posteriores. De este modo, una salida o un traslado se comprueba considerando el saldo resultante de los movimientos anteriores del mismo lote. Si se produce un error semántico, el lote completo se rechaza, incluso cuando algunos movimientos anteriores hayan sido válidos. Los cambios efectuados durante la evaluación permanecen únicamente en memoria.

El lote se acepta cuando supera las tres fases de validación y todos sus movimientos cumplen las reglas semánticas aplicables. En ese caso, el sistema prepara un JSON con los movimientos interpretados y el stock proyectado. En la ejecución por consola, este resultado puede guardarse como archivo JSON local; en la interfaz web, el JSON se presenta como parte de la respuesta en memoria. Si el lote se rechaza, no se genera un JSON de aceptación. En ninguno de los dos casos se modifica el archivo data/stock.json, porque el prototipo no confirma transacciones ni actualiza un inventario empresarial real.

Criterios generales

Criterio	Condición de aceptación	Condición de rechazo
Análisis léxico	No se detectan tokens auxiliares L01-L05.	Se identifica al menos un error léxico.
Análisis sintáctico	El documento cumple la gramática y contiene uno o más movimientos válidos en estructura.	Se identifica un incumplimiento estructural S01-S08.
Extracción	VisitorMovimientos obtiene los movimientos del árbol sintáctico válido.	No se ejecuta esta etapa cuando existe un error léxico o sintáctico.
Validación semántica	Todos los movimientos cumplen las reglas M01-M09.	Al menos un movimiento incumple una regla semántica.
Secuencia del lote	Cada movimiento se evalúa considerando los cambios previos de la copia temporal.	La secuencia produce una inconsistencia, como un saldo negativo.

Resultado	Se genera el resultado de aceptación y el JSON correspondiente.	Se presenta el rechazo y sus diagnósticos; no se genera JSON de aceptación.
Integridad del archivo inicial	data/stock.json conserva su contenido original.	El prototipo no contempla modificarlo; cualquier modificación sería ajena al comportamiento previsto.

Nota. Elaboración propia a partir de la gramática, el código de procesamiento y los criterios de validación implementados.

Tabla 14

Catálogo de errores léxicos L01-L05

Código	Condición	Fase	Diagnóstico	Ejemplo	Componente responsable
L01	Se encuentra un carácter que no coincide con las reglas léxicas definidas.	Léxica	Carácter no reconocido por el lenguaje.	CANTIDAD: 10$;	Regla ERROR_CARACTER de Inventario.g4 y diccionario LEXICAL_ERRORS.
L02	Se encuentra una cadena abierta sin comilla de cierre antes del fin de línea o del archivo.	Léxica	Cadena de texto sin comilla de cierre.	SKU: "HER-0001;	Regla ERROR_CADENA_SIN_CIERRE y diccionario LEXICAL_ERRORS.

L03	Se encuentra un número con parte decimal.	Léxica	Las cantidades deben expresarse mediante números enteros.	CANTIDAD: 10.5;	Regla ERROR_DECIMAL y diccionario LEXICAL_ERRORS.
L04	Se encuentra un número entero precedido por signo negativo.	Léxica	Las cantidades no admiten signo negativo.	CANTIDAD: -2;	Regla ERROR_ENTERO_NEGATIVO y diccionario LEXICAL_ERRORS.
L05	Se encuentra una cadena de texto vacía, delimitada por dos comillas dobles consecutivas.	Léxica	El valor de texto no puede estar vacío.	SKU: "";	Regla ERROR_CADENA_VACIA y diccionario LEXICAL_ERRORS.

Nota. Elaboración propia a partir de las reglas auxiliares del lexer y del diccionario LEXICAL_ERRORS implementado en Python.

Tabla 15

Catálogo de errores sintácticos S01-S08

Código	Condición	Fase	Diagnóstico	Ejemplo	Componente responsable
S01	Un campo aparece sin el separador : requerido entre el nombre y el valor.	Sintáctica	Falta el separador : entre el campo y su valor.	SKU "HER-0001";	InventarioParser y ListenerSintactico en src/main.py.
S02	Un campo no termina con el delimitador ;.	Sintáctica	Falta el terminador ; del campo.	CANTIDAD: 10 seguido de otro	InventarioParser y ListenerSintactico.

				campo dentro del mismo movimiento.	
S03	El documento termina sin la llave de cierre requerida para el bloque.	Sintáctica	El bloque no tiene la llave de cierre requerida.	LOTE { ENTRADA { ... } sin la llave final del lote.	InventarioParser y ListenerSintactico.
S04	Se omite un campo obligatorio de acuerdo con el tipo de movimiento y la posición esperada.	Sintáctica	Falta un campo obligatorio del movimiento.	Una ENTRADA sin UBICACION_DESTINO.	InventarioParser y ListenerSintactico.
S05	Se presenta un campo en una posición que no corresponde al orden definido para el movimiento.	Sintáctica	El campo aparece fuera del orden establecido.	UBICACION_DESTINO antes de CANTIDAD en una ENTRADA.	InventarioParser y ListenerSintactico.
S06	Se repite un campo que ya apareció en el mismo movimiento.	Sintáctica	El campo está duplicado dentro del movimiento.	Dos campos CANTIDAD en una misma instrucción.	ListenerSintactico, mediante el seguimiento de los campos vistos.
S07	Aparece un identificador en posición de campo, pero no corresponde a ninguno de los campos admitidos por el movimiento.	Sintáctica	El campo no pertenece a la estructura del movimiento.	ALMACEN: "A-01-03"; dentro de una ENTRADA.	IDENTIFICADOR, InventarioParser y ListenerSintactico.

S08	El lote contiene llaves de apertura y cierre, pero no incluye movimientos.	Sintáctica	El lote debe contener al menos un movimiento.	LOTE { }	Regla listaMovimientos de Inventario.g4 y ListenerSintactico.

Nota. Elaboración propia a partir de las producciones de Inventario.g4 y de la clasificación implementada en ListenerSintactico.

Tabla 16

Catálogo de errores semánticos M01-M09

Código	Condición	Fase	Diagnóstico	Ejemplo	Componente responsable
M01	El SKU de un movimiento no existe en el catálogo de productos.	Semántica	El producto indicado no está registrado.	SKU: "HER-9999"; cuando ese SKU no aparece en productos.json.	ValidadorSemantico y data/productos.json.
M02	Una ubicación requerida por el movimiento no existe en el catálogo de ubicaciones.	Semántica	La ubicación indicada no está registrada.	UBICACION_DESTINO: "Z-99-99"; cuando no aparece en ubicaciones.json.	ValidadorSemantico y data/ubicaciones.json.
M03	La cantidad del movimiento es igual a cero.	Semántica	La cantidad debe ser mayor que cero.	CANTIDAD: 0;	ValidadorSemantico, después del reconocimiento léxico de ENTERO.

M04	La operación dejaría el saldo de una ubicación por debajo de cero al considerar la secuencia del lote.	Semántica	Stock insuficiente para ejecutar el movimiento.	Una SALIDA de 20 unidades cuando el saldo disponible es 8.	ValidadorSemantico y copia temporal del stock.
M05	Un traslado utiliza la misma ubicación como origen y destino.	Semántica	El origen y el destino de un traslado deben ser diferentes.	UBICACION_ORIGEN: "A-01-03"; y UBICACION_DESTINO: "A-01-03";.	ValidadorSemantico y data/ubicaciones.json.
M06	La referencia documental no existe o no es compatible con el tipo de movimiento.	Semántica	La referencia no existe o no corresponde al movimiento.	Utilizar una referencia de salida en una ENTRADA.	ValidadorSemantico y data/referencias.json.
M07	El valor de TIPO_AJUSTE no corresponde a INCREMENTO ni DECREMENTO.	Semántica	El tipo de ajuste no está permitido.	TIPO_AJUSTE: DISMINUIR;	ValidadorSemantico, después del reconocimiento sintáctico del valor.
M08	El motivo utilizado en un ajuste no se encuentra entre los motivos autorizados.	Semántica	El motivo del ajuste no está autorizado.	MOTIVO: "Fiesta"; cuando ese valor no figura en motivos.json.	ValidadorSemantico y data/motivos.json.

M09	La cantidad del movimiento supera la cantidad documentada en la referencia asociada.	Semántica	La cantidad solicitada supera la cantidad documentada.	Solicitar 6 unidades cuando la referencia documenta 5.	ValidadorSemantico y data/referencias.json.

Nota. Elaboración propia a partir de las reglas M01-M09 documentadas en el README y de los diagnósticos observados en los casos de ejecución semántica.

1.5. Definición del alfabeto

El alfabeto del DSL comprende los caracteres que pueden intervenir en la escritura de sus instrucciones. Su definición debe considerar tanto los símbolos estructurales del lenguaje como los caracteres que pueden aparecer dentro de los valores textuales. La gramática Inventario.g4 establece las llaves, los dos puntos, el punto y coma y las comillas dobles como elementos estructurales; reconoce dígitos para las cantidades y letras mayúsculas, dígitos y guiones bajos para los identificadores. Además, permite valores textuales entre comillas dobles, siempre que no contengan comillas internas, tabulaciones ni saltos de línea.

Para expresar formalmente el conjunto de caracteres, se utiliza U como el conjunto finito de caracteres Unicode que puede procesar la entrada. Se define C como el conjunto de caracteres permitidos dentro de un valor textual, excluyendo las comillas dobles, el retorno de carro, el salto de línea y la tabulación. El alfabeto se representa de la siguiente manera:

$$\Sigma = \Sigma_L \cup \Sigma_D \cup \Sigma_I \cup \Sigma_S \cup \Sigma_T$$

Donde:

- $\Sigma_L = \{A, B, ..., Z\}$ representa las letras mayúsculas utilizadas en las palabras reservadas, los nombres de campos y los identificadores.
- $\Sigma_D = \{0, 1, ..., 9\}$ representa los dígitos utilizados en las cantidades y los identificadores.
- $\Sigma_I = \{_\}$ representa el guion bajo, admitido dentro de los identificadores.
- $\Sigma_S = \{espacio, tabulación, CR, LF, \{,\},:,;\}$ representa los espacios y delimitadores estructurales.
- $\Sigma_T = C \cup \{"\}$ representa los caracteres que pueden intervenir en los valores textuales y las comillas dobles que los delimitan.

En esta definición, $C = U \setminus \{", CR,\ LF,\ TAB\}$. Por tanto, los valores textuales pueden contener letras minúsculas, espacios, guiones, puntos y otros caracteres, siempre que no incluyan los caracteres excluidos. Esta amplitud corresponde a la regla CADENA de la gramática; no significa que esos caracteres puedan utilizarse libremente fuera de las cadenas entre comillas.

Tabla 12

Grupos de caracteres del alfabeto del DSL

Grupo	Conjunto o representación	Aplicación en el lenguaje	Ejemplos
Letras	A-Z	Palabras reservadas, nombres de campos e identificadores.	LOTE, ENTRADA, SKU
Dígitos	0-9	Cantidades enteras y componentes numéricos de identificadores.	10, 2026, A1
Guion bajo	_	Separación de segmentos dentro de identificadores.	TIPO_AJUSTE

Espacios permitidos	Espacio, tabulación, retorno de carro y salto de línea	Separación entre elementos; la gramática los descarta durante el análisis léxico.	Espacios entre campos y saltos de línea
Delimitadores	{, }, :, ;	Apertura y cierre de bloques, separación entre campo y valor, y terminación de campos.	LOTE {, SKU:, 10;
Comillas dobles	"	Delimitación de los valores de tipo cadena.	"HER-0001"
Caracteres de texto	C	Contenido de los valores textuales, excluyendo comillas dobles, tabulaciones y saltos de línea.	"Martillo de acero", "A-01-03"

Nota. Elaboración propia a partir de las clases de caracteres declaradas en grammar/Inventario.g4. La admisión de un carácter depende de la regla léxica en la que aparece.

Es necesario distinguir el alfabeto de los tokens y las palabras reservadas. El alfabeto está compuesto por caracteres individuales; los tokens representan categorías reconocidas por el lexer; y las palabras reservadas son secuencias específicas de caracteres que tienen un significado estructural dentro del DSL. Por ejemplo, ENTRADA está formada por letras del alfabeto, pero el lexer la reconoce como el token reservado ENTRADA, no como un identificador genérico.

Asimismo, los delimitadores poseen una función estructural y no deben confundirse con los caracteres que forman parte de los valores. El guion de "HER-0001" pertenece al contenido de una cadena, mientras que las llaves, los dos puntos y el punto y coma organizan la instrucción. La gramática también admite espacios y saltos de línea entre los elementos porque la regla WS los descarta durante el análisis léxico.

1.6. Definición de cadenas y lenguaje

Una cadena es una secuencia finita de caracteres pertenecientes al alfabeto Σ. El conjunto de todas las cadenas posibles sobre dicho alfabeto se representa mediante Σ^*, e incluye la cadena vacía ε, cuya longitud es cero. Sin embargo, no todas las cadenas de Σ^* pertenecen al lenguaje del DSL, porque deben cumplir las reglas léxicas y la estructura gramatical establecida.

El lenguaje del DSL se define como el conjunto de documentos que satisfacen la regla inicial documento de Inventario.g4. Esta regla exige un bloque LOTE, seguido de una llave de apertura, una lista no vacía de movimientos y una llave de cierre, sin contenido adicional después del lote. Por consiguiente, la cadena vacía no pertenece al lenguaje, ya que no contiene el bloque obligatorio ni los movimientos requeridos.

Formalmente, el lenguaje puede expresarse de la siguiente manera:

$$L(G) = \{w \in \Sigma^* \mid w\ satisface\ la\ gramática\ G\ del\ DSL\ de\ inventario\}$$

Donde G representa la gramática definida en Inventario.g4. La estructura general del documento puede resumirse como:

$$documento \rightarrow LOTE\ \{\ movimiento^+\ \}$$

El operador $+$ indica que debe existir al menos un movimiento dentro del lote. Cada movimiento debe corresponder a una de las cuatro alternativas reconocidas por la gramática: ENTRADA, SALIDA, TRASLADO o AJUSTE. Los campos que integran cada alternativa deben presentarse en el orden establecido y utilizar los delimitadores correspondientes.

Ejemplo de cadena válida en términos léxicos y sintácticos

LOTE {

```
  SALIDA {
    SKU: "HER-0001";
    CANTIDAD: 2;
    UBICACION_ORIGEN: "A-01-03";
    REFERENCIA: "SAL-2026-001";
  }
}
```

La instrucción anterior utiliza un movimiento SALIDA con los campos obligatorios y el orden definidos en la gramática. Sus valores también corresponden a los datos simulados del proyecto: el SKU está registrado, la ubicación existe y la referencia pertenece al tipo de movimiento indicado. La cantidad está dentro del límite documentado y no supera el stock inicial de diez unidades de ese producto en la ubicación indicada. Por ello, constituye un ejemplo válido para las tres fases de análisis con los datos de ejemplo disponibles.

Ejemplo de cadena léxicamente inválida

```
LOTE {
  ENTRADA {
    SKU: "HER-0001";
    CANTIDAD: 2.5;
    UBICACION_DESTINO: "A-01-03";
    REFERENCIA: "OC-2026-001";
  }
}
```

La cantidad 2.5 no corresponde al patrón de enteros definido para el token ENTERO. La gramática dispone de un token auxiliar para reconocer cantidades decimales y clasificarlas como error léxico L03. Por tanto, la entrada se rechaza en el análisis léxico y no continúa hacia la comprobación sintáctica.

Ejemplo de cadena sintácticamente inválida

```
LOTE {
```

```
  ENTRADA {
    SKU: "HER-0001";
    CANTIDAD: 2
    UBICACION_DESTINO: "A-01-03";
    REFERENCIA: "OC-2026-001";
  }
}
```

En este caso, la cantidad está escrita como entero y los demás elementos utilizan caracteres reconocibles. Sin embargo, falta el punto y coma que debe terminar el campo CANTIDAD. La secuencia de tokens no satisface la producción gramatical correspondiente, por lo que se clasifica como error sintáctico S02.

Ejemplo de cadena sintácticamente válida, pero semánticamente inválida

```
LOTE {
  SALIDA {
    SKU: "HER-0001";
    CANTIDAD: 20;
    UBICACION_ORIGEN: "A-01-03";
    REFERENCIA: "SAL-2026-002";
  }
}
```

La estructura cumple la gramática y sus valores textuales se encuentran correctamente delimitados. No obstante, el stock inicial de HER-0001 en A-01-03 es de diez unidades. Aunque la referencia SAL-2026-002 admite hasta veinte unidades, la salida solicitada supera las existencias disponibles y provoca el incumplimiento de M04. En consecuencia, el lote se rechaza en la fase semántica.

La distinción entre estos ejemplos permite identificar que una cadena puede estar formada por caracteres reconocibles y aun así incumplir la gramática, o puede cumplir la gramática y presentar una inconsistencia respecto de los datos del dominio. El lenguaje reconocido por el parser determina la

validez estructural, mientras que las reglas semánticas establecen si los movimientos pueden aceptarse de acuerdo con los catálogos y el stock simulado.

1.7. Expresiones regulares

Las expresiones regulares describen los patrones de caracteres que el analizador léxico reconoce como componentes del DSL. En la implementación, estos patrones se encuentran definidos en grammar/Inventario.g4 y son utilizados por el lexer generado mediante ANTLR 4.13.2. Para este proyecto, las expresiones más relevantes corresponden a los enteros, identificadores y cadenas de texto, además de las palabras reservadas y los delimitadores que estructuran los movimientos.

Tabla 13

Expresiones regulares de los componentes léxicos del DSL

Token o categoría	**Finalidad**	**Patrón en notación formal**	**Ejemplos válidos**	**Ejemplos inválidos**
ENTERO	Reconocer cantidades expresadas mediante dígitos sin signo ni parte decimal.	[0-9]+	0, 2, 10, 2026	-2, 2.5
IDENTIFICADOR	Reconocer identificadores compuestos por letras mayúsculas, dígitos y segmentos separados por guion bajo.	[A-Z][A-Z0-9]*(_[A-Z0-9]+)*	INCREMENTO, DECREMENTO, TIPO_AJUSTE, AJUSTE1	tipo, 1AJUSTE, TIPO_
CADENA	Reconocer valores textuales no vacíos delimitados por comillas dobles.	"[^"\r\n\t]+"	"HER-0001", "A-01-03", "Martillo de acero"	"", "HER-0001, una cadena con salto de línea

Palabras reservadas	Reconocer las estructuras y operaciones del DSL.	Literales exactos: LOTE, ENTRADA, SALIDA, TRASLADO, AJUSTE, INCREMENTO, DECREMENTO.	LOTE, ENTRADA, INCREMENTO	lote, Entrada, TRASLAD
Nombres de campos	Reconocer los campos admitidos por cada tipo de movimiento.	Literales exactos: SKU, CANTIDAD, UBICACION_ORIGEN, UBICACION_DESTINO, UBICACION, REFERENCIA, TIPO_AJUSTE, MOTIVO, OBSERVACION.	SKU, CANTIDAD, REFERENCIA	PRODUCTO, ALMACEN, CANT
Delimitadores	Reconocer los símbolos que estructuran los bloques y campos.	Literales exactos: \{, \}, :, ;.	{, }, :, ;	(,), ,
Espacios y saltos	Reconocer los separadores que se descartan durante el análisis.	[\t\r\n]+	Espacio, tabulación, salto de línea	No constituye una categoría de contenido de campo.

Nota. Elaboración propia a partir de las reglas del lexer de grammar/Inventario.g4. Los literales de palabras reservadas, campos y delimitadores se presentan como secuencias exactas y no como patrones generales de identificadores.

La expresión regular de ENTERO reconoce una o más cifras, por lo que admite léxicamente el valor 0. La condición de que la cantidad sea estrictamente mayor que cero no se impone en este patrón, sino mediante la regla semántica M03. De esta manera, se mantiene la diferencia entre el reconocimiento del formato numérico y la validación de su significado en el dominio.

Por otro lado, los valores de SKU, UBICACION_ORIGEN, UBICACION_DESTINO, UBICACION, REFERENCIA, MOTIVO y OBSERVACION utilizan el token general CADENA. La gramática no define una expresión regular independiente para cada uno de estos campos. La comprobación de si un

SKU, una ubicación, una referencia o un motivo existe y está autorizado corresponde a la fase semántica, que consulta los archivos JSON de datos maestros.

El campo TIPO_AJUSTE presenta una particularidad: la gramática admite los tokens INCREMENTO, DECREMENTO o un IDENTIFICADOR. Esta última alternativa permite que un valor con formato de identificador llegue a la fase semántica, donde M07 verifica que el contenido sea exactamente INCREMENTO o DECREMENTO. Por tanto, el reconocimiento léxico no sustituye la validación semántica del valor permitido.

1.8. Autómata finito no determinista (AFND)

Los autómatas finitos permiten representar formalmente los patrones descritos mediante expresiones regulares. Para el proyecto se seleccionaron tres componentes representativos de la gramática: el número entero, el identificador y la cadena de texto delimitada por comillas dobles. Esta selección permite estudiar una categoría numérica, una categoría alfanumérica con una estructura interna y una categoría textual con delimitadores de apertura y cierre.

Para mantener la correspondencia con la implementación, se utilizarán los patrones exactos de Inventario.g4. Es importante precisar que el patrón léxico de ENTERO admite el valor cero porque reconoce una o más cifras. La condición de cantidad positiva se comprueba posteriormente mediante M03. Por ello, el primer autómata reconoce enteros no negativos desde el punto de vista léxico, no exclusivamente enteros positivos.

1.8.1. AFND para números enteros

Expresión regular

$$R_1 = [0 - 9]^+$$

Donde $D = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. El operador $+$ indica que debe aparecer al menos un dígito.

Quíntupla formal

$$AFND_1 = (Q, \Sigma, \delta, q_0, F)$$

- $Q = \{q_0, q_1\}$
- $\Sigma = D$
- q_0 es el estado inicial.
- $F = \{q_1\}$
- $\delta(q_0, d) = \{q_1\}$, para todo $d \in D$.
- $\delta(q_1, d) = \{q_1\}$, para todo $d \in D$.

Descripción de estados

- q_0: estado inicial, todavía no se ha leído ningún dígito.
- q_1: estado de aceptación, se ha leído al menos un dígito y se pueden seguir leyendo más cifras.

Tabla 14

Transiciones del AFND para números enteros

Estado actual	Símbolo de entrada	Estado siguiente
q_0	D	q_1
q_1	D	q_1

1.8.2. Autómata finito determinista (AFD)

Expresión regular

$$R_2 = [A - Z][A - Z0 - 9]^*([A - Z0 - 9]^+)^*$$

Para simplificar la descripción de las transiciones, se definen:

- $A = \{A, B, ..., Z\}$: letras mayúsculas.
- $D = \{0, 1, ..., 9\}$: dígitos.
- $X = A \cup D$: letras mayúsculas o dígitos.
- _: guion bajo.

La expresión exige que el identificador comience con una letra mayúscula. Después puede contener letras o dígitos y, opcionalmente, segmentos que comienzan con guion bajo y contienen al menos una letra o un dígito.

Quíntupla formal

$$AFND_2 = (Q, \Sigma, \delta, q_0, F)$$

- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = A \cup D \cup \{_\}$
- q_0 es el estado inicial.
- $F = \{q_1, q_3\}$
- Las transiciones se definen mediante la siguiente función:

$$\delta(q_0, a) = \{q_1\},\ a \in A$$

$$\delta(q_1, x) = \{q_1\},\ x \in A$$

$$\delta(q_1, _) = \{q_2\}$$

$$\delta(q_2, x) = \{q_3\},\ x \in X$$

$$\delta(q_3, x) = \{q_3\},\ x \in X$$

$$\delta(q_3, _) = \{q_2\}$$

Las transiciones no especificadas se consideran vacías, es decir, no conducen a otro estado de aceptación.

Descripción de estados

- q_0: estado inicial; exige una letra mayúscula.
- q_1: estado de aceptación; representa un identificador que contiene al menos una letra y puede continuar con letras o dígitos.
- q_2: estado intermedio; se ha leído un guion bajo y se exige al menos una letra o un dígito.
- q_3: estado de aceptación; representa un segmento posterior al guion bajo y permite continuar con letras, dígitos o nuevos segmentos.

Tabla 15

Transiciones del AFND para identificadores

Estado actual	**Símbolo de entrada**	**Estado siguiente**
q_0	A	q_1
q_1	$A \cup D$	q_1
q_1	_	q_2
q_2	$A \cup D$	q_3

q_3	$A \cup D$	q_3
q_3	–	q_2

Elaboración propia .

1.8.3. AFND para cadenas de texto

Expresión regular

\[R_3 = " [^"\r\n\t]^+ " \]

$$R_3 = \text{ " } [\text{ " } \backslash r \backslash n \backslash t] \ ^+$$

Se define el conjunto C como el conjunto de caracteres que pueden aparecer dentro de una cadena de texto, excluyendo:

- Comillas dobles: ".
- Retorno de carro: \r.
- Salto de línea: \n.
- Tabulación: \t.

Por tanto, la cadena debe comenzar con una comilla doble, contener al menos un carácter permitido y terminar con otra comilla doble.

Quíntupla formal

$$AFND_3 = (Q, \Sigma, \delta, q_0, F)$$

- $Q = \{q_0, q_1, q_2, q_3\}$

- $\Sigma = C \cup \{"\}$
- q_0 es el estado inicial.
- $F = \{q_3\}$
- Las transiciones se definen como:

$$\delta(q_0, x) = \{q_1\}$$

$$\delta(q_1, c) = \{q_2\},\ c \in C$$

$$\delta(q_2, c) = \{q_2\},\ c \in C$$

$$\delta(q_2, ") = \{q_3\}$$

Descripción de estados

- q_0: estado inicial; espera la comilla doble de apertura.
- q_1: se ha reconocido la comilla de apertura y se exige al menos un carácter de contenido.
- q_2: se ha reconocido uno o más caracteres de contenido; puede continuar leyendo caracteres o recibir la comilla de cierre.
- q_3: estado final; se ha reconocido una cadena completa.

Tabla 16

Transiciones del AFND para cadenas de texto

Estado actual	**Símbolo de entrada**	**Estado siguiente**
q_0	"	q_1
q_1	C	q_2

q_2	C	q_2
q_2	"	q_3

Nota: Elaboración propia

1.9. Autómata finito determinista

1.9.1. AFD para números enteros

El AFND para enteros utiliza los estados q_0 y q_1, y sus transiciones están definidas únicamente para los dígitos del conjunto D. Como desde cada estado existe una sola transición para esa clase de símbolos, la construcción por subconjuntos genera los siguientes estados:

$$Q' = \{\{q_0\}, \{q_1\}\}$$

Se renombran los conjuntos para facilitar la lectura:

- $A_0 = \{q_0\}$: estado inicial.
- $A_1 = \{q_1\}$: estado de aceptación.

No se necesita un estado pozo porque el alfabeto del autómata está restringido a los dígitos del cero al nueve.

Quíntupla formal

$$AFD_1 = (Q', \Sigma, \delta', A_0, F')$$

- $Q' = \{A_0, A_1\}$
- $\Sigma = D = \{0, 1, \ldots, 9\}$
- Estado inicial: A_0

- Estados finales: $F' = \{A_1\}$

La función de transición es:

$$\delta'(A_0, d) = A_1,\ d \in D$$

$$\delta'(A_1, d) = A_1,\ d \in D$$

Tabla 17

Transiciones del AFD para números enteros

Estado actual	Símbolo de entrada	Estado siguiente
A_0	D	A_1
A_1	D	A_1

Nota: Elaboración propia

Estados de aceptación

El único estado de aceptación es A_1, porque representa una secuencia que contiene al menos un dígito.

El estado A_0 no es final, de modo que la cadena vacía no se acepta.

Comparación con el AFND

Aspecto	AFND	AFD
Estados	q_0, q_1	A_0, A_1

Estado inicial	q_0	$A_0 = \{q_0\}$
Estado final	q_1	$A_1 = \{q_1\}$
Transición con dígitos	Una transición por estado	Una transición única por estado
Lenguaje reconocido	$[0 - 9]^+$	$[0 - 9]^+$

La conversión conserva el patrón [0-9]+. Por consiguiente, el autómata acepta 0, 7 y 125, pero no acepta -2, 2.5 ni una cadena vacía. La restricción de que la cantidad sea mayor que cero continúa correspondiendo a M03 y no al autómata léxico.

1.9.2. AFD para identificadores

El AFND de identificadores utiliza cuatro estados. Como cada clase de entrada conduce a un único estado siguiente, los conjuntos generados por el método de subconjuntos son unitarios:

$$Q' = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}\}$$

Para completar la función de transición del AFD sobre el alfabeto definido, se incorpora el estado pozo ⊥, que recibe las entradas no permitidas desde cada estado.

Se utilizan las siguientes clases:

- A: letras mayúsculas.
- D: dígitos.
- _: guion bajo.
- ⊥: estado pozo para transiciones no válidas.

Los estados resultantes se renombran:

- $B_0 = \{q_0\}$: inicial.
- $B_1 = \{q_1\}$: final.
- $B_2 = \{q_2\}$: intermedio.
- $B_3 = \{q_3\}$: final.
- $\perp$: estado pozo.

Quíntupla formal

$$AFD_2 = (Q', \Sigma, \delta', B_0, F')$$

- $Q' = \{B_0, B_1, B_2, B_3, \perp\}$
- $\Sigma = A \cup D \cup \{_\}$
- Estado inicial: B_0
- Estados finales: $F' = \{B_1, B_3\}$

La función de transición se define mediante las siguientes reglas:

$$\delta'(B_0, a) = B_1,\ a \in A$$

$$\delta'(B_0, d) = \perp,\ d \in D$$

$$\delta'(B_0, _) = \perp$$

$$\delta'(B_1, x) = B_1,\ x \in A \cup D$$

$$\delta'(B_1, _) = B_2$$

$$\delta'(B_2, x) = B_3,\ x \in A \cup D$$

$$\delta'\ (B_2, _) = \perp$$

$$\delta'\ (B_3, x) = B_3,\ x \in A \cup D$$

$$\delta'\ (B_3, _) = B_2$$

$$\delta'\ (\perp, s) = \perp,\ s \in \Sigma$$

Tabla 18

Transiciones del AFD para identificadores

Estado actual	**Letras A-Z**	**Dígitos 0-9**	**Guion bajo _**
B_0	B_1	$\perp$	$\perp$
B_1	B_1	B_1	B_2
B_2	B_3	B_3	$\perp$
B_3	B_3	B_3	B_2
$\perp$	$\perp$	$\perp$	$\perp$

Nota: Elaboración propia

Estados de aceptación

Los estados finales son B_1 y B_3. El primero acepta identificadores que no contienen guion bajo, mientras que el segundo acepta identificadores que terminan en un segmento posterior al guion bajo.

El estado B_2 no es final porque representa una secuencia que termina inmediatamente después del guion bajo.

Comparación con el AFND

Aspecto	AFND	AFD
Estados principales	q_0, q_1, q_2, q_3	B_0, B_1, B_2, B_3
Estado adicional	No requiere	$\perp$, para completar las transiciones
Estado inicial	q_0	$B_0 = \{q_0\}$
Estados finales	q_1, q_3	B_1, B_3
Lenguaje reconocido	[A-Z][A-Z0-9]*(_[A-Z0-9]+)*	[A-Z][A-Z0-9]*(_[A-Z0-9]+)*

La conversión conserva la estructura del identificador. Por ejemplo, INCREMENTO, AJUSTE1 y TIPO_AJUSTE son aceptados por el autómata, mientras que tipo, 1AJUSTE y TIPO_ son rechazados. El estado pozo permite representar las entradas que no pueden continuar formando un identificador válido.

Debe considerarse que el autómata reconoce el patrón general de IDENTIFICADOR. Cuando un lexema coincide exactamente con una palabra reservada, ANTLR aplica la prioridad de las reglas del lexer y asigna el token reservado correspondiente. Por ello, el reconocimiento del patrón de identificador no significa que palabras como ENTRADA o INCREMENTO se clasifiquen siempre como identificadores.

1.9.3. AFD para cadenas de texto

El AFND de cadenas de texto utiliza cuatro estados. La construcción por subconjuntos produce los siguientes conjuntos unitarios:

$$Q' = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3\}\}$$

Se renombran los estados:

- $C_0 = \{q_0\}$: inicial.
- $C_1 = \{q_1\}$: espera contenido.
- $C_2 = \{q_2\}$: contiene al menos un carácter.
- $C_3 = \{q_3\}$: final.

Para completar la función de transición se incorpora el estado pozo $\perp$. El alfabeto del autómata se divide en dos clases: C, que representa los caracteres permitidos dentro del texto, y la comilla doble ", que actúa como delimitador.

Quíntupla formal

$$AFD_3 = (Q', \Sigma, \delta', C_0, F')$$

- $Q' = \{C_0, C_1, C_2, C_3, \perp\}$
- $\Sigma = C \cup \{"\}$
- Estado inicial: C_0
- Estados finales: $F' = \{C_3\}$

La función de transición es:

$$\delta'\ (C_0, ") = C_1$$

$$\delta'\ (C_0, c) = \perp,\ c \in C$$

$$\delta'\ (C_1, c) = C_2,\ c \in C$$

$$\delta'\ (C_1, '') = \perp$$

$$\delta'\ (C_2, c) = C_2,\ c \in C$$

$$\delta'\ (C_2, '') = C_3$$

$$\delta'\ (C_3, s) = \perp,\ s \in \Sigma$$

$$\delta'\ (\perp, s) = \perp,\ s \in \Sigma$$

Tabla 19

Transiciones del AFD para cadenas de texto

Estado actual	**Comilla doble "**	**Carácter permitido** C
C_0	C_1	$\perp$
C_1	$\perp$	C_2
C_2	C_3	C_2
C_3	$\perp$	$\perp$
$\perp$	$\perp$	$\perp$

Nota: Elaboración propia

Estados de aceptación

El único estado final es C_3, que se alcanza después de reconocer la comilla de apertura, al menos un carácter permitido y la comilla de cierre. Los estados C_0, C_1, C_2 y $\perp$ no son finales.

Comparación con el AFND

Aspecto	AFND	AFD
Estados principales	q_0, q_1, q_2, q_3	C_0, C_1, C_2, C_3
Estado adicional	No requiere	$\perp$, para completar las transiciones
Estado inicial	q_0	$C_0 = \{q_0\}$
Estado final	q_3	C_3
Contenido mínimo	Al menos un carácter de C	Al menos un carácter de C
Lenguaje reconocido	"[^"\r\n\t]+"	"[^"\r\n\t]+"

La conversión conserva la condición de que la cadena tenga contenido y se encuentre delimitada por comillas dobles. Por tanto, el autómata acepta "HER-0001" y "Martillo de acero", pero rechaza "", una cadena sin comilla de cierre y cualquier cadena que contenga dentro de sus delimitadores los caracteres excluidos por la regla CADENA.

2. DISEÑO

2.1. Diseño de tokens y lexemas

El diseño léxico establece las categorías que permiten reconocer los componentes del DSL y distinguir las instrucciones, los nombres de campos, los valores y los delimitadores. Cada token

representa una categoría y cada lexema corresponde al texto concreto que es reconocido durante el análisis.

Tabla 17

Tokens principales del DSL de movimientos de inventario

Categoría	Token	Lexema de ejemplo	Función
Palabra reservada	LOTE	LOTE	Identifica el bloque principal del documento.
Palabra reservada	ENTRADA	ENTRADA	Identifica el ingreso de productos.
Palabra reservada	SALIDA	SALIDA	Identifica la salida de productos.
Palabra reservada	TRASLADO	TRASLADO	Identifica el movimiento entre ubicaciones.
Palabra reservada	AJUSTE	AJUSTE	Identifica una operación de ajuste.
Palabra reservada	INCREMENTO	INCREMENTO	Representa un ajuste que aumenta existencias.
Palabra reservada	DECREMENTO	DECREMENTO	Representa un ajuste que reduce existencias.
Nombre de campo	CAMPO_SKU	SKU	Identifica el código del producto.

Nombre de campo	CAMPO_CANTIDAD	CANTIDAD	Identifica la cantidad del movimiento.
Nombre de campo	CAMPO_UBICACION_ORIGEN	UBICACION_ORIGEN	Identifica la ubicación de origen.
Nombre de campo	CAMPO_UBICACION_DESTINO	UBICACION_DESTINO	Identifica la ubicación de destino.
Nombre de campo	CAMPO_REFERENCIA	REFERENCIA	Identifica el documento de respaldo.
Nombre de campo	CAMPO_TIPO_AJUSTE	TIPO_AJUSTE	Identifica la modalidad del ajuste.
Nombre de campo	CAMPO_UBICACION	UBICACION	Identifica la ubicación asociada al ajuste.
Nombre de campo	CAMPO_MOTIVO	MOTIVO	Identifica el motivo del ajuste.
Nombre de campo	CAMPO_OBSERVACION	OBSERVACION	Identifica el campo complementario opcional.
Literal	CADENA	"HER-0001"	Reconoce valores de texto delimitados por comillas.
Literal	ENTERO	10	Reconoce cantidades numéricas enteras sin signo.
Literal	IDENTIFICADOR	DISMINUIR	Reconoce un identificador válido, utilizado en la producción del tipo de ajuste.

Delimitador	LLAVE_ABRE	{	Marca el inicio de un bloque.
Delimitador	LLAVE_CIERRA	}	Marca el cierre de un bloque.
Delimitador	DOS_PUNTOS	:	Separa el nombre de un campo de su valor.
Delimitador	PUNTO_COMA	;	Finaliza la declaración de un campo.
Token auxiliar de error	ERROR_CADENA_VACIA	""	Permite diagnosticar una cadena vacía.
Token auxiliar de error	ERROR_CADENA_SIN_CIERRE	"HER-0001	Detecta una cadena sin comilla de cierre.
Token auxiliar de error	ERROR_DECIMAL	10.5	Identifica cantidades con parte decimal.
Token auxiliar de error	ERROR_ENTERO_NEGATIVO	-2	Identifica cantidades negativas.
Token auxiliar de error	ERROR_CARACTER	@	Detecta caracteres no reconocidos por las reglas anteriores.

Nota. Los tokens auxiliares permiten identificar entradas que deben diagnosticarse como errores léxicos; no forman parte de las producciones válidas del parser. Elaboración propia a partir de grammar/Inventario.g4.

2.2. Diseño de la gramática libre de contexto

La gramática del DSL se representa mediante $G = (V, T, P, S)$, donde V contiene los símbolos no terminales, T reúne los tokens terminales reconocidos por el lexer, P comprende las reglas de

producción y *S* representa el símbolo inicial. En el proyecto, *S* corresponde a documento, que contiene un lote completo.

Los terminales comprenden las palabras reservadas, los nombres de campos, los literales y los delimitadores definidos en el diseño léxico. Los no terminales representan el documento, el lote, la lista de movimientos, los cuatro tipos de operación y sus campos. Las producciones se implementaron en grammar/Inventario.g4, que constituye la especificación fuente utilizada por ANTLR para generar el analizador sintáctico.

La gramática establece un único bloque LOTE por documento, con uno o más movimientos. Cada tipo de movimiento posee una secuencia propia de campos obligatorios y admite OBSERVACION al final, de manera opcional. La gramática comprueba la estructura y el orden de los elementos, mientras que las condiciones relacionadas con los catálogos y las existencias se verifican en la fase semántica.

2.3. Reglas de producción

Las producciones definitivas se presentan a continuación siguiendo la estructura declarada en Inventario.g4. Se emplea la notación de ANTLR: + representa una o más repeticiones y ? representa un elemento opcional.

Tabla 18

Producciones definitivas de la gramática del DSL

Regla	Símbolo no terminal	Producción
R1	documento	lote EOF
R2	lote	LOTE LLAVE_ABRE listaMovimientos LLAVE_CIERRA
R3	listaMovimientos	movimiento+

R4	movimiento	entrada | salida | traslado | ajuste
R5	entrada	ENTRADA LLAVE_ABRE campoSku campoCantidad campoUbicacionDestino campoReferencia campoObservacion? LLAVE_CIERRA
R6	salida	SALIDA LLAVE_ABRE campoSku campoCantidad campoUbicacionOrigen campoReferencia campoObservacion? LLAVE_CIERRA
R7	traslado	TRASLADO LLAVE_ABRE campoSku campoCantidad campoUbicacionOrigen campoUbicacionDestino campoReferencia campoObservacion? LLAVE_CIERRA
R8	ajuste	AJUSTE LLAVE_ABRE campoSku campoCantidad campoTipoAjuste campoUbicacion campoMotivo campoReferencia campoObservacion? LLAVE_CIERRA
R9	campoSku	CAMPO_SKU DOS_PUNTOS CADENA PUNTO_COMA
R10	campoCantidad	CAMPO_CANTIDAD DOS_PUNTOS ENTERO PUNTO_COMA
R11	campoUbicacionOrigen	CAMPO_UBICACION_ORIGEN DOS_PUNTOS CADENA PUNTO_COMA
R12	campoUbicacionDestino	CAMPO_UBICACION_DESTINO DOS_PUNTOS CADENA PUNTO_COMA
R13	campoUbicacion	CAMPO_UBICACION DOS_PUNTOS CADENA PUNTO_COMA
R14	campoReferencia	CAMPO_REFERENCIA DOS_PUNTOS CADENA PUNTO_COMA

R15	campoTipoAjuste	CAMPO_TIPO_AJUSTE DOS_PUNTOS tipoAjuste PUNTO_COMA
R16	tipoAjuste	"INCREMENTO" | "DECREMENTO" | IDENTIFICADOR
R17	campoMotivo	CAMPO_MOTIVO DOS_PUNTOS CADENA PUNTO_COMA
R18	campoObservacion	CAMPO_OBSERVACION DOS_PUNTOS CADENA PUNTO_COMA

Nota. La producción R16 permite reconocer estructuralmente los valores INCREMENTO, DECREMENTO o un IDENTIFICADOR. Cuando se reconoce un identificador diferente de los dos valores admitidos, la entrada puede continuar hasta la validación semántica, donde se aplica M07. Elaboración propia a partir de grammar/Inventario.g4.

2.4. Derivaciones

Para demostrar la aplicación de las producciones se utiliza un movimiento ENTRADA con sus campos obligatorios y el campo opcional OBSERVACION. La derivación se realiza por la izquierda, sustituyendo en cada paso el no terminal más próximo al inicio.

Cadena terminal esperada:

```
LOTE {
  ENTRADA {
    SKU: "HER-0001";
    CANTIDAD: 10;
    UBICACION_DESTINO: "A-01-03";
    REFERENCIA: "OC-2026-001";
    OBSERVACION: "Recepcion de mercaderia";
  }
}
```

Derivación por la izquierda:

1. documento

2. ⇒ lote EOF
3. ⇒ LOTE LLAVE_ABRE listaMovimientos LLAVE_CIERRA EOF
4. ⇒ LOTE LLAVE_ABRE movimiento+ LLAVE_CIERRA EOF
5. ⇒ LOTE LLAVE_ABRE entrada LLAVE_CIERRA EOF
6. ⇒ LOTE LLAVE_ABRE ENTRADA LLAVE_ABRE campoSku campoCantidad campoUbicacionDestino campoReferencia campoObservacion? LLAVE_CIERRA LLAVE_CIERRA EOF
7. Se sustituyen los campos por sus producciones correspondientes:
    - campoSku ⇒ CAMPO_SKU DOS_PUNTOS CADENA PUNTO_COMA
    - campoCantidad ⇒ CAMPO_CANTIDAD DOS_PUNTOS ENTERO PUNTO_COMA
    - campoUbicacionDestino ⇒ CAMPO_UBICACION_DESTINO DOS_PUNTOS CADENA PUNTO_COMA
    - campoReferencia ⇒ CAMPO_REFERENCIA DOS_PUNTOS CADENA PUNTO_COMA
    - campoObservacion ⇒ CAMPO_OBSERVACION DOS_PUNTOS CADENA PUNTO_COMA
8. Se sustituyen los tokens de valor por sus lexemas concretos: "HER-0001", 10, "A-01-03", "OC-2026-001" y "Recepcion de mercaderia".

La derivación concluyó al obtener la cadena terminal presentada. El marcador EOF representa el final de la entrada y no se escribe como parte del archivo DSL. Las derivaciones de los demás tipos de movimiento pueden incorporarse como material complementario en los anexos.

2.5. Árbol sintáctico

El árbol sintáctico correspondiente a la misma instrucción representa la estructura reconocida por la gramática. Su raíz es documento; debajo se encuentran los nodos lote, listaMovimientos y

movimiento, que conducen a la producción entrada. Esta última se ramifica en los campos campoSku, campoCantidad, campoUbicacionDestino, campoReferencia y campoObservacion.

Los nodos terminales contienen los tokens y lexemas concretos de la instrucción, como ENTRADA, SKU, "HER-0001", CANTIDAD, 10 y los valores de ubicación y referencia. La estructura coincide con la derivación por la izquierda, porque ambas representan la aplicación de las mismas producciones. En la implementación, ANTLR genera el árbol de análisis a partir de la regla inicial documento, con nodos de contexto asociados a las reglas de la gramática y nodos terminales correspondientes a los tokens reconocidos.

El diseño formal definió la representación de los movimientos y las reglas necesarias para reconocer su estructura. Sobre esta especificación se desarrolló la automatización de los analizadores y su integración con la validación semántica.

3. DESARROLLO

3.1. Automatización mediante metacompiladores

Se seleccionó ANTLR 4.13.2 para automatizar la generación de los analizadores a partir de una única especificación gramatical. El archivo Inventario.g4 definió las reglas léxicas y sintácticas del DSL, mientras que el generador produjo los módulos encargados de reconocer tokens, comprobar la estructura y permitir el recorrido del árbol sintáctico.

La generación incluyó el lexer, el parser y el visitor para Python. Este último permitió recorrer el árbol reconocido y convertir sus nodos en estructuras de datos que pudieran ser procesadas por el validador semántico. De este modo, la implementación mantuvo separadas la especificación formal del lenguaje y las reglas de negocio programadas en Python.

3.2. Generación del analizador léxico y sintáctico

Los componentes principales de la automatización se organizaron de la siguiente manera:

Tabla 19

Componentes principales de la generación y el procesamiento

Archivo o componente	Tipo	Función
grammar/Inventario.g4	Manual	Contiene las reglas léxicas y sintácticas del DSL.
generated/InventarioLexer.py	Generado	Reconoce los tokens de la entrada.
generated/InventarioParser.py	Generado	Comprueba la estructura mediante las producciones de la gramática.
generated/InventarioVisitor.py	Generado	Proporciona las operaciones para recorrer el árbol sintáctico.
src/visitor_movimientos.py	Manual	Transforma el árbol en movimientos estructurados de Python.
src/validador_semantico.py	Manual	Comprueba las reglas semánticas y proyecta los efectos sobre una copia temporal del stock.
src/main.py	Manual	Coordina la ejecución del programa por consola.
app/routes.py	Manual	Gestiona las solicitudes de la interfaz web Flask y presenta los resultados.

Nota. Los módulos generados se encuentran versionados en el repositorio. La regeneración requiere Java y el archivo JAR de ANTLR 4.13.2. Elaboración propia a partir de la estructura del proyecto.

El comando documentado para regenerar los módulos desde la carpeta raíz del repositorio fue:

java -jar .\antlr-4.13.2-complete.jar -Dlanguage=Python3 -visitor -no-listener -Xexact-output-dir -o .\generated .\grammar\Inventario.g4

3.3. Implementación de la validación semántica

Después del análisis sintáctico, VisitorMovimientos recorrió el árbol y construyó objetos de movimiento con los datos de cada instrucción. Estos objetos fueron enviados a ValidadorSemantico, que consultó los archivos JSON de productos, ubicaciones, referencias, motivos y existencias iniciales.

El validador aplicó las reglas M01-M09 para comprobar la existencia de los productos y ubicaciones, la cantidad, la disponibilidad de stock, la diferencia entre origen y destino, la compatibilidad de las referencias y las condiciones de los ajustes. El procesamiento fue secuencial: cada movimiento válido se proyectó sobre una copia temporal del inventario, de modo que las operaciones posteriores consideraran los saldos resultantes de las anteriores.

Cuando se detectó una inconsistencia semántica, el lote completo fue rechazado y no se generó un resultado de aceptación. Cuando todos los movimientos superaron las comprobaciones, se construyó una salida JSON con las operaciones interpretadas y el stock proyectado. En ambos casos, el archivo original data/stock.json permaneció sin modificaciones.

3.4. Integración de los componentes

La integración conectó las etapas de reconocimiento y validación en un flujo único. La entrada DSL pasó primero por el analizador léxico; si no se detectaron errores, sus tokens fueron procesados por el parser. El árbol sintáctico resultante fue recorrido por VisitorMovimientos, que produjo los movimientos estructurados para la validación semántica. Finalmente, el sistema aceptó el lote o presentó los diagnósticos encontrados.

Flujo general:

Entrada DSL → Lexer → Tokens → Parser → Árbol sintáctico → VisitorMovimientos → Movimientos estructurados → Validador semántico → Aceptación o diagnósticos.

Para representar la integración en el cuerpo del trabajo se utilizará una sola figura de arquitectura general. Los diagramas de secuencia y el flujo técnico detallado se conservarán como material complementario.

3.5. Implementación de la interfaz de usuario

Se incorporó una interfaz web mediante Flask para facilitar la ejecución del prototipo. El usuario puede escribir las instrucciones DSL directamente en el formulario o cargar un archivo con extensión .dsl o .txt. Al procesar la entrada, la aplicación presenta los tokens reconocidos, el árbol sintáctico, los movimientos extraídos, los diagnósticos y el estado final del lote.

La interfaz utiliza el mismo flujo de validación que la aplicación por consola. La diferencia corresponde a la presentación y al manejo de la salida: en consola, un lote aceptado puede generar un archivo JSON en el directorio local salidas/; en la aplicación web, el JSON se presenta directamente en pantalla y no se escribe en dicho directorio. La interfaz web tampoco modifica data/stock.json.

3.6. Despliegue de la aplicación

El código fuente se mantuvo bajo control de versiones mediante Git y se publicó en GitHub, lo que permitió conservar la versión del proyecto y conectar el repositorio con el servicio de alojamiento. Para ejecutar la aplicación web se configuró Gunicorn como servidor WSGI y se realizó el despliegue en Render.

La aplicación quedó disponible como demostración pública en *https://inventario-dsl.onrender.com/* . Este despliegue permitió acceder a la interfaz sin iniciar manualmente el servidor local, aunque su disponibilidad y capacidad quedaron sujetas a las condiciones y límites del servicio de alojamiento. El despliegue no representó una instalación empresarial ni una integración con sistemas reales: el prototipo continuó trabajando con datos simulados, sin conexión a una base de datos corporativa ni actualización de existencias reales.

El desarrollo integró los analizadores generados, el procesamiento semántico y las dos modalidades de ejecución. La siguiente etapa evaluó su comportamiento mediante casos de regresión y verificaciones funcionales, sin atribuir a estas pruebas un alcance de evaluación empresarial.

4. PRUEBAS

4.1. Estrategia y casos de prueba

La evaluación se realizó mediante casos manuales de integración y regresión preparados para comprobar el comportamiento del prototipo ante entradas válidas y entradas con errores léxicos, sintácticos o semánticos. Los escenarios se ejecutaron desde PowerShell y se comparó el código de salida obtenido con el esperado: 0 para los lotes aceptados y 1 para los rechazados.

La estrategia permitió verificar el reconocimiento de las instrucciones, el cumplimiento de la gramática, la aplicación de las reglas semánticas y la política de rechazo integral. También se comprobó que la proyección de los movimientos no modificara el archivo original de existencias. Las evidencias se conservaron mediante capturas de ejecución y los archivos de prueba del repositorio.

Herramienta de automatización seleccionada

Se seleccionó Selenium como herramienta propuesta para automatizar pruebas de la interfaz web, debido a que el prototipo utiliza Flask y permite interactuar con formularios, cargar archivos y visualizar resultados desde un navegador. Selenium permitiría reproducir las acciones de un usuario y comprobar elementos de la página, como el estado de aceptación, los diagnósticos y el resultado JSON.

Sin embargo, no se encontró evidencia de una implementación o ejecución de pruebas con Selenium. Por tanto, su selección se documenta como una decisión técnica pendiente de implementación y no como una actividad ya ejecutada. La evaluación presentada en este documento corresponde a los casos manuales realizados desde PowerShell.

4.2. Ejecución de pruebas

La regresión local comprendió 31 escenarios distribuidos en cuatro categorías. La ejecución comparó el código de salida real de cada archivo con el resultado esperado y registró que los 31 casos finalizaron con el código correspondiente, sin fallos en esa comprobación.

Tabla 20

Resumen de los casos de prueba ejecutados

Categoría	Cantidad	Finalidad	Resultado general	Evidencia
Entradas válidas	6	Comprobar la aceptación de los movimientos y lotes admitidos.	Código de salida esperado.	Casos de casos/validos/ y capturas de aceptación.
Errores léxicos	5	Comprobar la detección de las condiciones L01-L05.	Código de rechazo esperado.	Casos de casos/lexicos/ y capturas léxicas.
Errores sintácticos	8	Comprobar la detección de las condiciones S01-S08.	Código de rechazo esperado.	Casos de casos/sintacticos/ y capturas sintácticas.
Errores semánticos	12	Comprobar las reglas M01-M09, el saldo acumulado y el rechazo integral.	Código de salida esperado para cada escenario, incluido el caso válido de saldo acumulado.	Casos de casos/semanticos/ y capturas semánticas.
Total	31	Verificar el comportamiento integrado del prototipo.	31 casos con código de salida esperado y ninguno fallido en esta ejecución.	Matriz de pruebas y evidencias seleccionadas.

Nota. La categoría semántica incluye un escenario de saldo acumulado válido, por lo que no todos los casos de dicha carpeta corresponden a rechazos. Elaboración propia a partir de la regresión documentada en el repositorio.

4.3. Verificación de resultados y comportamiento del sistema

La verificación de requisitos se resumió según el comportamiento observable del prototipo. Los requisitos cuyo detalle no quedó identificado de forma inequívoca en la versión documental disponible se mantienen pendientes de cotejo con la matriz consolidada, sin asignarles un cumplimiento no demostrado.

Tabla 21

Resumen de verificación de requisitos funcionales y no funcionales

Grupo de requisitos	Cumplidos	Parciales	Pendientes	Evidencia principal
RF01-RF02: ingreso de instrucciones por texto o archivo	2	0	0	Interfaz Flask y carga de archivos .dsl y .txt.
RF03-RF06: análisis y generación de diagnósticos	4	0	0	Tokens, árbol sintáctico y diagnósticos de las tres fases.
RF07-RF08: generación y presentación de resultados	1	1	0	JSON generado en consola y resultado mostrado en la interfaz web; la salida web no se guarda como archivo local.
RF09-RF10: rechazo integral y consulta de datos maestros	2	0	0	Validación sobre copia temporal y archivos JSON del caso.
RF11-RF12	0	0	2	Pendiente de cotejo con la descripción definitiva de ambos requisitos.

RNF01, RNF03-RNF05: claridad, integridad, modularidad y reproducibilidad	4	0	0	Diagnósticos, separación de componentes y casos fijos de regresión.
RNF02 y RNF06: trazabilidad y facilidad de uso	0	2	0	Resultados visibles en la interfaz; no se documentó una evaluación con usuarios ni una bitácora persistente.
RNF07-RNF09	0	0	3	Pendiente de cotejo con la descripción definitiva de estos requisitos.

Nota. La clasificación de RF11-RF12 y RNF07-RNF09 queda sujeta a la correspondencia con la matriz final de requisitos. No se asignó un resultado sin disponer de su descripción verificable. Elaboración propia.

La evaluación tuvo límites importantes. El caso de estudio fue hipotético y los datos empleados fueron simulados; no se contó con registros empresariales reales ni con participación de usuarios de una organización. Las pruebas fueron principalmente manuales y no incluyeron mediciones de carga, rendimiento o escalabilidad. Tampoco se ejecutó Selenium ni se realizó una comparación experimental entre un procedimiento convencional y el validador formal. En consecuencia, la regresión comprobó el comportamiento esperado de los escenarios preparados, pero no permitió estimar una mejora estadística ni un impacto organizacional.

RESULTADOS

Los resultados se presentan en el orden de los cinco objetivos específicos. Se distinguen los productos desarrollados, las verificaciones efectivamente ejecutadas y los objetivos para los cuales todavía no se obtuvo evidencia suficiente.

Tabla 22

Resultados obtenidos según los objetivos específicos

Objetivo	Resultado obtenido	Evidencia	Interpretación
OE1. Identificar los errores de los movimientos de inventario.	Se organizó un catálogo de errores léxicos L01-L05, sintácticos S01-S08 y semánticos M01-M09.	Tablas 14, 15 y 16 del documento.	Se diferenciaron los problemas de representación, estructura y coherencia empresarial.
OE2. Definir las reglas de reconocimiento de los movimientos.	Se formalizaron los tokens, los lexemas y las producciones de la gramática del DSL.	Tablas 17 y 18, archivo Inventario.g4, derivación y árbol sintáctico.	Se estableció una especificación que permitió reconocer los cuatro tipos de movimiento y sus campos.
OE3. Implementar la validación formal.	Se integraron ANTLR 4.13.2, los módulos generados, VisitorMovimientos, ValidadorSemantico, la ejecución por consola y la interfaz Flask.	Tabla 19, diagrama general de arquitectura y capturas de ejecución.	Se obtuvo un prototipo funcional que procesó lotes y separó las tres fases de validación.
OE4. Evaluar el comportamiento mediante casos de prueba.	Se ejecutaron 31 escenarios manuales: 6 válidos, 5 léxicos, 8 sintácticos y 12 semánticos. Los códigos de salida coincidieron con los esperados.	Tabla 20, matriz de pruebas y capturas seleccionadas.	La regresión comprobó el comportamiento previsto para los escenarios preparados, sin medir rendimiento ni precisión en un entorno real.
OE5. Comparar la detección con y sin validación formal.	No se ejecutó una comparación controlada con una línea base convencional ni se obtuvo una medición comparativa.	No existe una matriz comparativa ni una medición experimental verificable.	No fue posible establecer cuánto cambió la detección frente a un procedimiento sin validación formal.

Nota. Los resultados describen productos y ejecuciones verificables del prototipo. No representan mediciones de impacto en una empresa real. Elaboración propia.

La principal evidencia técnica obtenida fue la integración de un lenguaje formal con analizadores generados, una pasada semántica y una interfaz de demostración. Asimismo, la prueba de regresión confirmó que los escenarios preparados devolvieron los códigos de salida esperados. La validación por lote completo evitó generar una salida de aceptación cuando se detectaron errores y mantuvo sin modificaciones el archivo data/stock.json.

En conjunto, los resultados demostraron la viabilidad técnica del prototipo para reconocer y validar los movimientos simulados. No demostraron una reducción cuantificada de errores empresariales, porque no se dispuso de una muestra real ni de una evaluación comparativa con un procedimiento convencional.

DISCUSIÓN

El prototipo obtuvo un mecanismo que reconoció la estructura de los movimientos de inventario y comprobó su coherencia mediante reglas léxicas, sintácticas y semánticas. Este resultado se relacionó con Destro et al. (2023), quienes analizaron cómo la inexactitud de los registros de inventario afectaba el desempeño de los centros de distribución y evaluaron el conteo cíclico como medida de control. Ambos trabajos abordaron la fiabilidad de la información, aunque desde momentos diferentes del proceso: el estudio de Destro et al. examinó los efectos operativos de las discrepancias y una forma de corregirlas mediante controles posteriores, mientras que el prototipo se concentró en rechazar determinadas inconsistencias antes de generar una salida de aceptación. Esta diferencia mostró que la validación formal podía complementar los controles físicos, pero no reemplazarlos. Un analizador no podía reconocer por sí solo diferencias entre el stock registrado y las existencias físicas ni detectar errores originados exclusivamente en actividades de recepción o picking. Además, el antecedente empleó simulación de dinámica de sistemas y el proyecto utilizó datos ficticios y pruebas funcionales manuales. Por ello, la coincidencia estuvo en la preocupación por la fiabilidad de los registros, pero no en el método ni en el alcance de evaluación. Se concluyó que el prototipo abordó una parte específica del problema: la consistencia lógica de las instrucciones antes de su procesamiento.

Corella-Parra y Olea-Miranda (2023) desarrollaron un sistema de control de inventario que combinó diagnóstico de procesos, clasificación ABC, controles internos, código de barras y software para organizar el flujo de materiales. En su estudio se reportaron cambios en la satisfacción del cliente y en el tiempo de preparación de pedidos, como resultado de la intervención conjunta sobre la gestión del almacén. El presente proyecto también incorporó automatización, pero su intervención fue más acotada: definió un DSL para representar entradas, salidas, traslados y ajustes, y añadió validaciones formales a los datos introducidos. La diferencia principal radicó en que el antecedente modificó prácticas y herramientas de gestión dentro de una empresa comercializadora, mientras que el prototipo se desarrolló sobre un caso hipotético y no administró el inventario real de una organización. Por tanto, sus resultados no fueron comparables en términos de satisfacción, tiempo o eficiencia operativa. La experiencia de Corella-Parra y Olea-Miranda mostró la importancia de combinar tecnología y controles organizativos; el prototipo aportó una especificación computacional que podría situarse antes del registro, sin reemplazar las prácticas de almacenamiento, clasificación y verificación. Se concluyó que ambas aproximaciones respondieron a problemas relacionados, pero actuaron sobre niveles distintos del proceso y requirieron evidencias diferentes para determinar su utilidad.

Jara-Saenz et al. (2026) estudiaron la integración de pronósticos de demanda, cantidad económica de pedido y logística esbelta para gestionar inventarios en el sector peruano de productos de limpieza. Su enfoque se dirigió a la planificación y optimización de las existencias mediante técnicas cuantitativas y simulación, mientras que el prototipo desarrollado se concentró en verificar la validez formal y semántica de cada movimiento. En consecuencia, ambos trabajos utilizaron recursos computacionales para abordar problemas de inventario, pero no evaluaron la misma variable ni persiguieron el mismo resultado. El antecedente buscó mejorar indicadores relacionados con los días de inventario, la rotación y los costos de almacenamiento; el proyecto no incorporó pronósticos, políticas de reposición ni modelos de optimización. Asimismo, el artículo presentó una discrepancia entre los días iniciales descritos en sus conclusiones y los consignados en una de sus tablas, por lo que no se utilizó esa cifra como base de comparación directa. La diferencia indicó que una operación correctamente estructurada no necesariamente representa una decisión óptima de abastecimiento. El validador pudo comprobar

restricciones definidas, pero no determinar por sí mismo cuánto comprar ni cuándo reponer. Se concluyó que la validación formal y la planificación de inventarios podían ser complementarias, aunque el prototipo no demostró mejoras en los indicadores logísticos estudiados por Jara-Saenz et al.

García Peña et al. (2023) examinaron la relación entre el control interno de inventario y la gestión de resultados en negocios comerciales de la región de San Martín. Mediante un diseño cuantitativo, correlacional y transversal, reportaron una relación positiva entre ambas variables. El prototipo coincidió con el antecedente en la importancia asignada a las reglas de control, especialmente en la autorización de motivos de ajuste, la existencia de referencias documentarias y la conservación de la integridad del lote. Sin embargo, el alcance fue distinto: García Peña et al. estudiaron prácticas de control interno en negocios, mientras que el proyecto implementó reglas informáticas sobre movimientos simulados. La validación de M01-M09 no midió la calidad general del control interno ni permitió establecer una relación estadística con los resultados de una empresa. Su principal aporte fue convertir determinadas restricciones operativas en condiciones verificables por un programa, lo que permitió identificar errores antes de producir una salida de aceptación. La limitación consistió en que las reglas dependieron de catálogos y datos ficticios, sin comprobar si reflejaban todas las políticas de una organización. Se concluyó que el prototipo podía representar un mecanismo técnico de apoyo al control interno, pero no constituía una evaluación integral ni una demostración de la relación entre control y desempeño empresarial.

Vasquez-Layza et al. (2024) implementaron estrategias de control de inventarios y gestión de almacenes en una empresa ferretera peruana, mediante herramientas como Kardex, clasificación ABC, rediseño del layout y capacitación. El antecedente utilizó mediciones antes y después de la intervención y reportó cambios en los costos operacionales. En contraste, el presente proyecto se limitó a validar la representación de los movimientos y las reglas asociadas con productos, ubicaciones, referencias y existencias. Ambos trabajos se relacionaron con el control de inventario en un entorno comercial y reconocieron la importancia de contar con información organizada, pero sus intervenciones fueron diferentes. Mientras el estudio de Vasquez-Layza et al. actuó directamente sobre procedimientos de almacén y evaluó resultados operativos, el prototipo procesó archivos DSL y

simuló el efecto secuencial de las operaciones sobre una copia del stock. La comparación permitió reconocer que un mecanismo formal podría reducir la aceptación de instrucciones incoherentes, pero no podía reorganizar físicamente el almacén, capacitar al personal ni reducir costos por sí mismo. Además, las pruebas del prototipo no utilizaron una muestra empresarial ni mediciones previas y posteriores. Se concluyó que la solución desarrollada representó un componente informático potencialmente complementario a las estrategias de gestión, sin evidencia para atribuirle los resultados operativos del antecedente.

Floriano Rodríguez et al. (2024) analizaron la relación entre el control interno y la gestión de inventarios en micro y pequeñas empresas comerciales de Chimbote. Su investigación utilizó un diseño correlacional y encuestas aplicadas a negocios, y encontró una asociación positiva entre las variables, además de identificar limitaciones en la gestión de inventarios. El proyecto coincidió con ese antecedente en la preocupación por el registro y el control de los productos, pero adoptó una perspectiva técnica más delimitada. Las reglas del DSL verificaron que los movimientos tuvieran una estructura reconocible y cumplieran condiciones de coherencia antes de generar el resultado JSON. No obstante, la validación no evaluó la capacitación del personal, las políticas de compras, el pronóstico de demanda ni la gestión integral de las MYPE. La diferencia también se reflejó en los instrumentos: el antecedente recopiló información de empresas, mientras que el prototipo se verificó mediante escenarios programados y datos simulados. Por ello, no fue posible comparar estadísticamente sus resultados ni afirmar que el sistema mejorara la eficiencia de las organizaciones estudiadas. El análisis mostró que una herramienta formal podía apoyar una dimensión específica del control, pero su utilidad organizacional requería integrarse con procedimientos y datos reales. Se concluyó que el prototipo aportó una alternativa de validación preventiva que debía evaluarse en condiciones empresariales antes de atribuirle efectos sobre la gestión de inventarios.

CONCLUSIONES

1. Se identificaron y organizaron los errores que podían presentarse durante el procesamiento de los movimientos de inventario. La clasificación permitió diferenciar las inconsistencias léxicas, sintácticas y semánticas mediante los catálogos definidos para el DSL.
2. Se definieron los tokens, lexemas y producciones de la gramática libre de contexto para representar los movimientos de entrada, salida, traslado y ajuste. La especificación quedó implementada en Inventario.g4 y sirvió como base para generar los analizadores.
3. Se implementó un prototipo que integró ANTLR 4.13.2, el recorrido del árbol sintáctico mediante VisitorMovimientos y la validación semántica con Python. La solución incorporó ejecución por consola e interfaz web mediante Flask.
4. Se ejecutaron 31 escenarios manuales de regresión y todos devolvieron el código de salida esperado. Las pruebas permitieron comprobar el comportamiento previsto para entradas válidas y para los errores léxicos, sintácticos y semánticos incluidos en la matriz.
5. No se obtuvo una medición comparativa entre un procedimiento convencional y la validación formal. Por ello, los resultados permitieron concluir que el prototipo funcionó técnicamente sobre los escenarios simulados, pero no demostrar una mejora cuantificada en la detección de inconsistencias de una empresa real.

RECOMENDACIONES

1. Implementar pruebas automatizadas con Selenium para verificar la carga de archivos, el envío de instrucciones y la presentación de resultados en la interfaz Flask. La automatización deberá incluir escenarios de aceptación y rechazo.
2. Evaluar el prototipo con datos reales autorizados y anonimizados de una empresa comercial, previa definición de las condiciones de acceso, confidencialidad y protección de la información.

3. Ampliar la matriz de pruebas con combinaciones de movimientos, referencias documentarias, cantidades y saldos acumulados, incluyendo escenarios límite y entradas mal formadas que permitan comprobar el comportamiento de las distintas fases.
4. Incorporar mediciones de rendimiento con diferentes tamaños de lote para estimar los tiempos de análisis, el consumo de recursos y el comportamiento del validador ante cargas crecientes.
5. Estudiar una integración segura con un sistema empresarial de inventario, manteniendo la validación como una etapa previa a la confirmación de operaciones y evitando que una prueba o un rechazo alteren directamente los registros productivos.
6. Mantener sincronizados Inventario.g4, los módulos generados, la documentación de tokens y producciones, y las reglas semánticas. Toda modificación deberá acompañarse de una regeneración controlada y de la ejecución de la matriz de regresión correspondiente.

REFERENCIAS

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, techniques, and tools (2nd ed.). Addison-Wesley.

Chomsky, N. (1956). Three models for the description of language. IRE Transactions on Information Theory, 2(3), 113–124.

Chomsky, N. (1959). On certain formal properties of grammars. Information and Control, 2(2), 137–167.

Corella-Parra, L. M., & Olea-Miranda, J. (2023). Desarrollo de un sistema de control de inventario para una empresa comercializadora de sistemas de riego. Ingeniería Investigación y Tecnología, 24(1), 1–10. https://doi.org/10.22201/fi.25940732e.2023.24.1.006

Destro, I. R., Staudt, F. H., Somensi, K., & Taboada, C. (2023). The impacts of inventory record inaccuracy and cycle counting on distribution center performance. Production, 33, Article e20220077. https://doi.org/10.1590/0103-6513.20220077

Fawcett, T. (2006). [DATOS BIBLIOGRÁFICOS PENDIENTES DE VERIFICACIÓN]

Floriano Rodríguez, R., Espejo Chacón, L. F., Farías Añazco, K. F., Carrasco Espinoza, M. E., & Corcino Cutamanca, H. D. (2024). Correlational analysis of internal control and inventory management in commercial SMEs of Chimbote. Edelweiss Applied Science and Technology, 8(6), 572–581. https://doi.org/10.55214/25768484.v8i6.2129

García Peña, M., López Ocmin, L. S., & Romero-Carazas, R. (2023). Control interno de inventario y la gestión de resultados de un emporio comercial de la región de San Martín – Perú. Región Científica, 2(2), Article 202392. https://doi.org/10.58763/rc202392

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). Introduction to automata theory, languages, and computation (3rd ed.). Pearson.

Jara-Saenz, L., Venegas-Chambergo, E., & Quiroz-Flores, J. C. (2026). Data-driven inventory management and lean logistics integration: A case study in the Peruvian household cleaning products sector. AIP Conference Proceedings, 3381, Article 090006. https://doi.org/10.1063/5.0308469

Johnson, S. C. (1975). Yacc: Yet another compiler-compiler (Computing Science Technical Report No. 32). Bell Laboratories.

Knuth, D. E. (1968). Semantics of context-free languages. Mathematical Systems Theory, 2(2), 127–145.

Lesk, M. E., & Schmidt, E. (1975). Lex: A lexical analyzer generator (Computing Science Technical Report No. 39). Bell Laboratories.

McNemar, Q. (1947). [DATOS BIBLIOGRÁFICOS PENDIENTES DE VERIFICACIÓN]

Parr, T. (2013). The definitive ANTLR 4 reference. Pragmatic Bookshelf.

Parr, T., & Fisher, K. (2011). LL(*): The foundation of the ANTLR parser generator. En Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (pp. 425–436). ACM.

Parr, T., Harwell, S., & Fisher, K. (2014). Adaptive LL(*): The power of dynamic analysis. En Proceedings of OOPSLA 2014 (pp. 579–598). ACM.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.

Royce, W. W. (1970). Managing the development of large software systems. En Proceedings of IEEE WESCON (pp. 1–9).

Sassaman, L., Patterson, M. L., Bratus, S., & Locasto, M. E. (2013). Security applications of formal language theory. IEEE Systems Journal, 7(3), 489–500.

Schwaber, K., & Sutherland, J. (2020). The Scrum Guide. https://scrumguides.org

Vasquez-Layza, L. N., Cerna Rios, N. B., & Escurra-Lagos, J. C. (2024). Implementation of inventory control and warehouse management strategies to reduce operational costs in a Peruvian hardware company. Proceedings of the 4th LACCEI International Multiconference on Entrepreneurship, Innovation and Regional Development (LEIRD 2024). https://doi.org/10.18687/LEIRD2024.1.1.420

ANEXOS

Anexo A. Gramática Inventario.g4.

Contiene el archivo fuente completo de ANTLR con las reglas léxicas, las producciones sintácticas y los tokens auxiliares de diagnóstico.

Anexo B. AFND y AFD adicionales.

Reúne las representaciones y tablas complementarias de los autómatas utilizados para el reconocimiento de los componentes léxicos.

Anexo C. Derivaciones y árboles sintácticos complementarios.

Incluye derivaciones y árboles de los movimientos de salida, traslado y ajuste, además del ejemplo de entrada presentado en el cuerpo.

Anexo D. Matriz completa de casos de prueba.

Presenta los 31 escenarios, sus entradas, categoría, condición evaluada y resultado esperado y obtenido.

Anexo E. Capturas de errores léxicos.

Contiene evidencias seleccionadas de la detección de caracteres no reconocidos, cadenas incompletas, cantidades decimales, cantidades negativas y cadenas vacías.

Anexo F. Capturas de errores sintácticos.

Reúne ejemplos de errores de estructura detectados por el parser, con sus diagnósticos correspondientes.

Anexo G. Capturas de errores semánticos.

Presenta evidencias de validaciones relacionadas con productos, ubicaciones, referencias, ajustes y disponibilidad de existencias.

Anexo H. Ejecución por consola.

Incluye capturas de la ejecución local, la aceptación de lotes, el rechazo de entradas y la generación de resultados JSON en el directorio de salida.

Anexo I. Interfaz web.

Contiene capturas de la aplicación Flask, la carga de archivos, la presentación de tokens, el árbol sintáctico, los movimientos extraídos y los resultados.

Anexo J. GitHub y Render.

Reúne evidencias del repositorio, la publicación del código, el estado del servicio y el acceso a la demostración pública.

Anexo K. Diagramas técnicos y fuentes PlantUML.

Contiene los diagramas de arquitectura, flujo de validación y secuencia para lotes aceptados y rechazados, junto con sus archivos fuente .puml.

Anexo L. Comprobación de integridad de data/stock.json.

Presenta la evidencia de que el archivo original de existencias permaneció sin modificaciones durante las validaciones del prototipo.